



**Politecnico
di Torino**

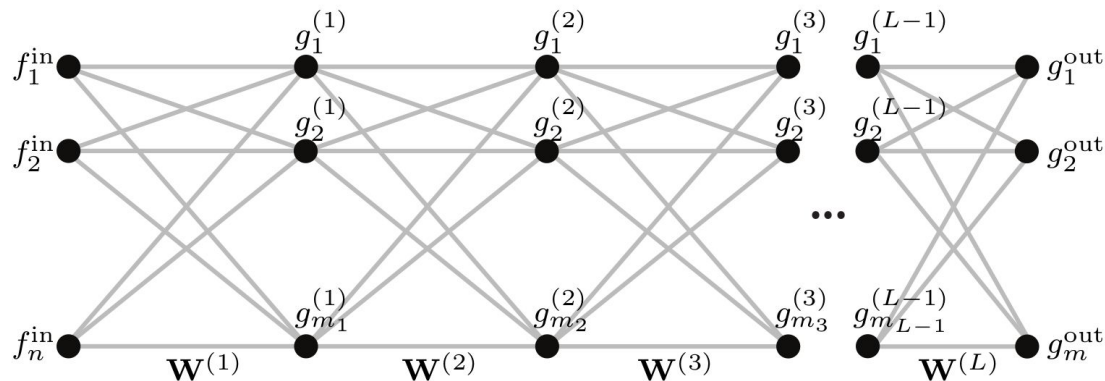


Advanced Machine Learning

A.A. 2025/2026

Tatiana Tommasi

Neural Network



Deep neural network consisting of L layers

Net output $\mathbf{g}^{\text{out}} = \sigma(\dots \mathbf{W}^{(2)} \sigma(\mathbf{W}^{(1)} \mathbf{f}^{\text{in}}))$

Activation, e.g. $\sigma(x) = \max\{x, 0\}$ rectified linear unit (ReLU)

Parameters weights of all layers $\mathbf{W}^{(1)}, \dots, \mathbf{W}^{(L)}$ (including biases)

The need of priors

Deep feed-forward networks are provably **universal**.

However:

- We can make them **arbitrarily complex**.
- The number of **parameters** can be huge.
- Very difficult to **optimize**.
- Very difficult to achieve **generalization**.

The need of priors

Deep feed-forward networks are provably **universal**.

However:

- We can make them **arbitrarily complex**.
- The number of **parameters** can be huge.
- Very difficult to **optimize**.
- Very difficult to achieve **generalization**.

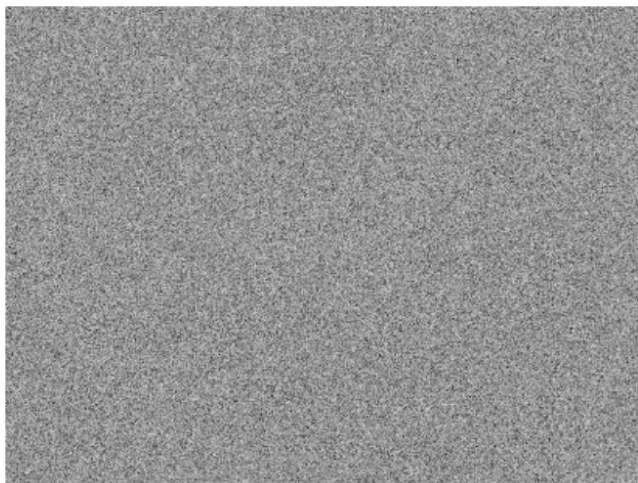
We need additional **priors** as a (partial) remedy to the above.

Look for “universal” priors that are **task-independent** to some extent.

Task-independent priors must come with the **data**.

Structure as a Strong Prior

Key insight: Data often carries **structural priors** in terms of repeating patterns, compositionality, locality, ...



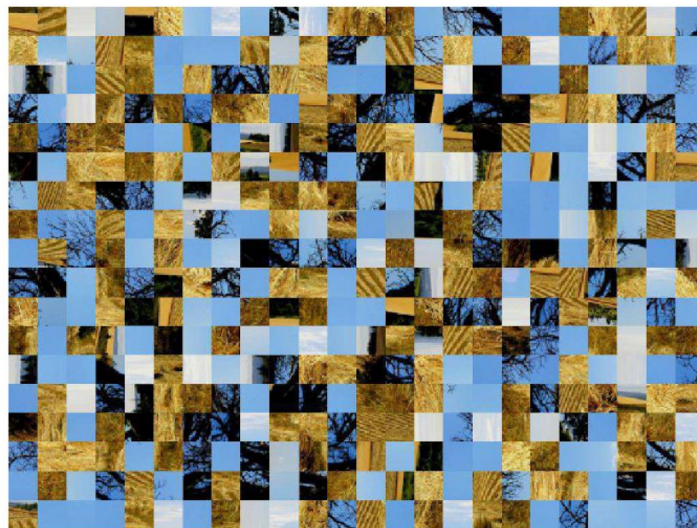
Structure as a Strong Prior

Key insight: Data often carries **structural priors** in terms of repeating patterns, compositionality, locality, ...



Structure as a Strong Prior

Key insight: Data often carries **structural priors** in terms of repeating patterns, compositionality, locality, ...



Son et al, "Solving Small-piece Jigsaw Puzzles by Growing Consensus", CVPR 2016

Structure as a Strong Prior

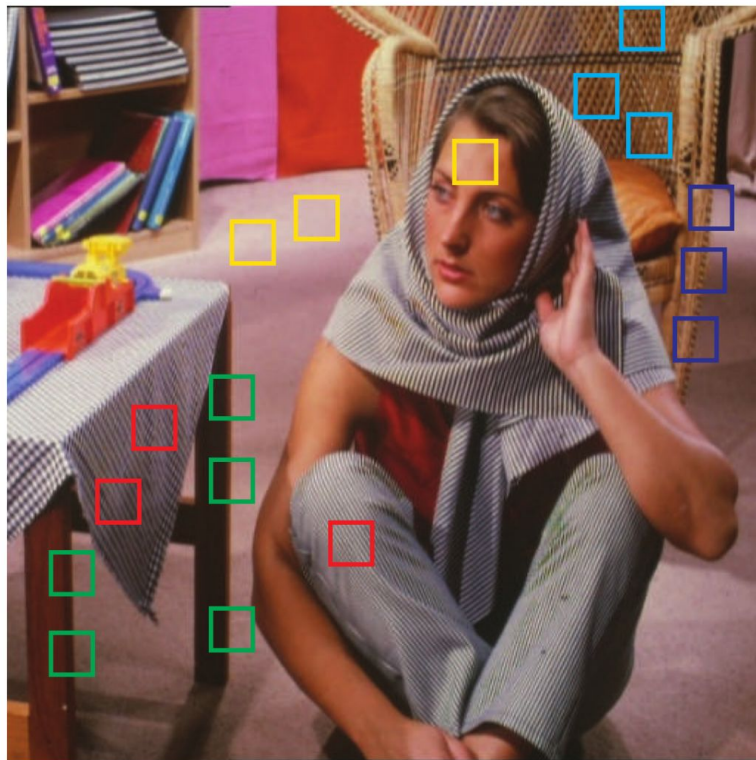
Key insight: Data often carries **structural priors** in terms of repeating patterns, compositionality, locality, ...



Son et al, "Solving Small-piece Jigsaw Puzzles by Growing Consensus", CVPR 2016

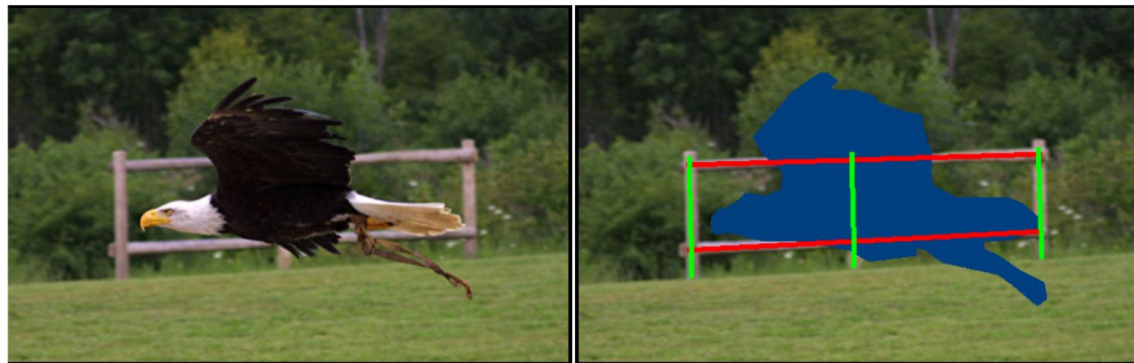
Self-Similarity

Data tends to be **self-similar** across the domain:



Self-Similarity

Data tends to be **self-similar** across the domain:



Barnes et al, "PatchMatch: A Randomized Correspondence Algorithm for Structural Image Editing", TOG 2009

Self-Similarity

Data tends to be [self-similar](#) across the domain:



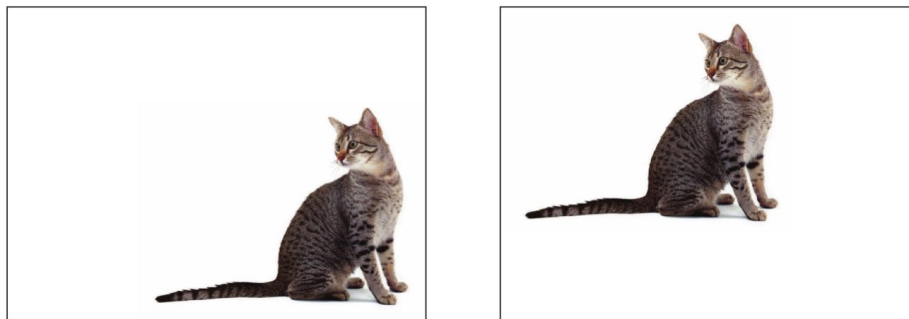
Barnes et al, "PatchMatch: A Randomized Correspondence Algorithm for Structural Image Editing", TOG 2009

Translation Invariance

Translations do not change the image content.

Define the (linear!) **translation operator** $\mathcal{T}$ along vector $v \in \mathbb{R}^2$ as:

$$\mathcal{T}_v f(x) = f(x - v)$$



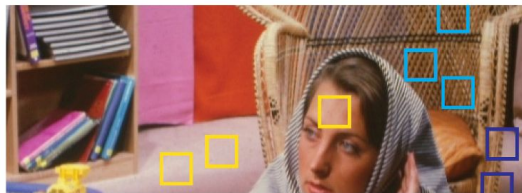
Therefore, it is desirable to enforce **translation invariance**:

$$y(\mathcal{T}_v f) = y(f) \quad \forall f, \mathcal{T}_v$$

where y is a classification functional.

Hierarchy and Compositionality

Translation invariance is desirable **across multiple scales**:



We expect **local features** to be invariant to their location in the image:

$$z(\mathcal{T}_v p) = z(p) \quad \forall p, \mathcal{T}_v$$

where p are image patches of variable size.



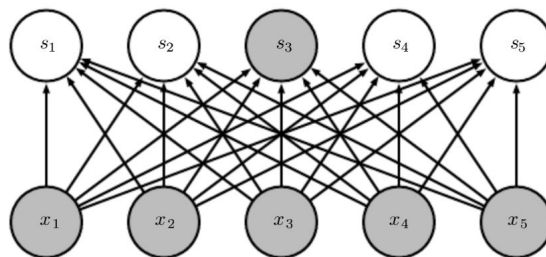
Convolutional Neural Networks

Data is often composed of hierarchical, local, shift-invariant patterns.

CNNs directly exploit this fact as a prior.

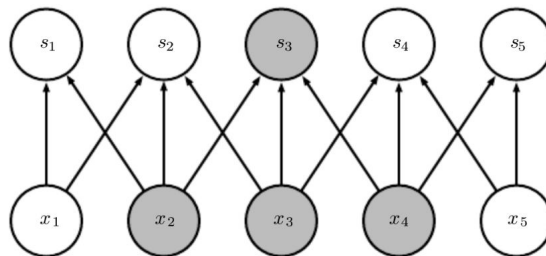
Sparse Interaction

Fully-connected layer:



each edge is a different weight

Convolutional layer:



the outgoing edges have the same weights
for each input variable (**weight sharing**)

Convolution

Given two functions $f, g : [-\pi, \pi] \rightarrow \mathbb{R}$ their **convolution** is a function:

$$(f \star g)(x) = \int_{-\pi}^{\pi} f(t)g(x-t)dt$$

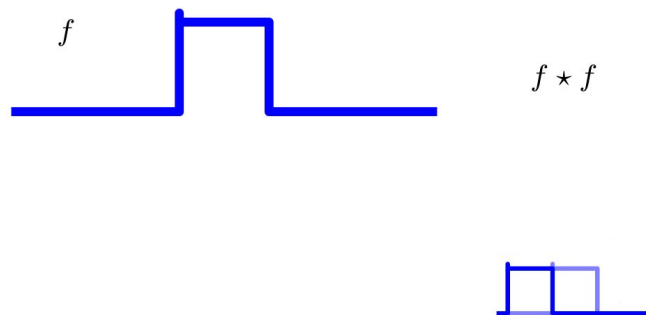


d'Alembert 1754

Convolution

Given two functions $f, g : [-\pi, \pi] \rightarrow \mathbb{R}$ their **convolution** is a function:

$$(f \star g)(x) = \int_{-\pi}^{\pi} f(t)g(x-t)dt$$

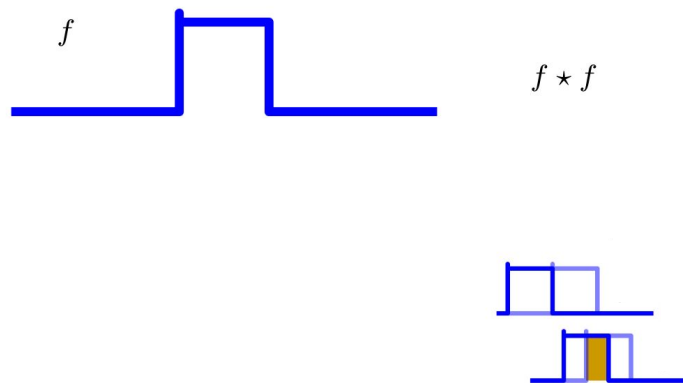


d'Alembert 1754

Convolution

Given two functions $f, g : [-\pi, \pi] \rightarrow \mathbb{R}$ their **convolution** is a function:

$$(f \star g)(x) = \int_{-\pi}^{\pi} f(t)g(x-t)dt$$

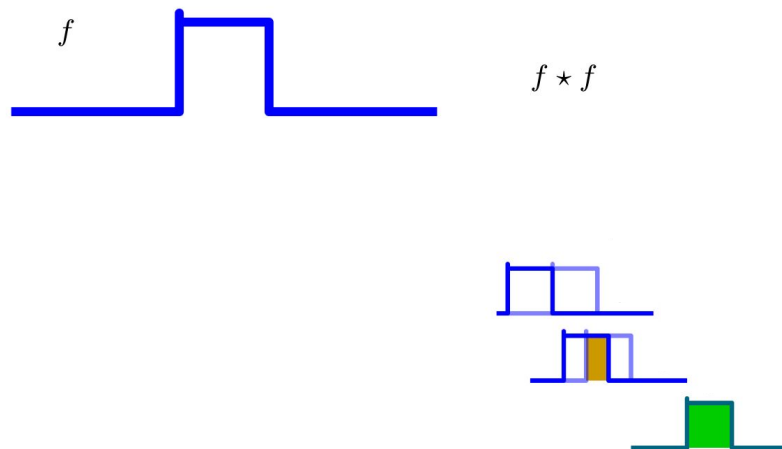


d'Alembert 1754

Convolution

Given two functions $f, g : [-\pi, \pi] \rightarrow \mathbb{R}$ their **convolution** is a function:

$$(f \star g)(x) = \int_{-\pi}^{\pi} f(t)g(x-t)dt$$

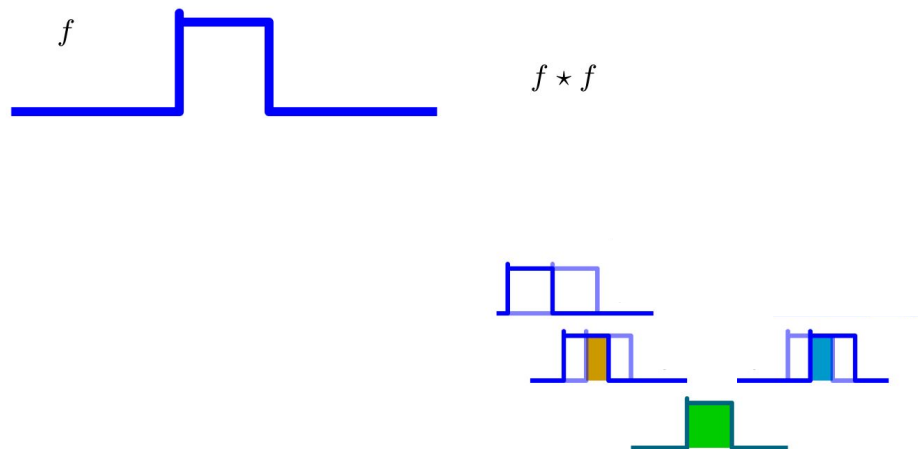


d'Alembert 1754

Convolution

Given two functions $f, g : [-\pi, \pi] \rightarrow \mathbb{R}$ their **convolution** is a function:

$$(f \star g)(x) = \int_{-\pi}^{\pi} f(t)g(x-t)dt$$

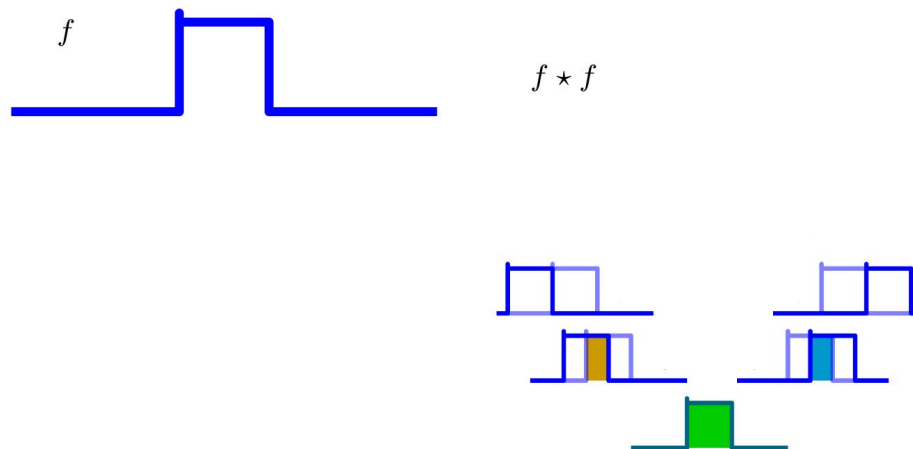


d'Alembert 1754

Convolution

Given two functions $f, g : [-\pi, \pi] \rightarrow \mathbb{R}$ their **convolution** is a function:

$$(f \star g)(x) = \int_{-\pi}^{\pi} f(t)g(x-t)dt$$

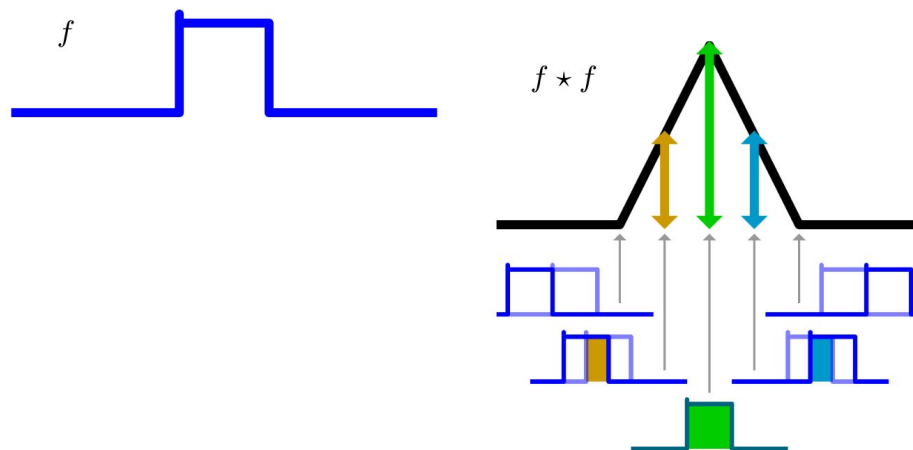


d'Alembert 1754

Convolution

Given two functions $f, g : [-\pi, \pi] \rightarrow \mathbb{R}$ their **convolution** is a function:

$$(f \star g)(x) = \int_{-\pi}^{\pi} f(t)g(x-t)dt$$

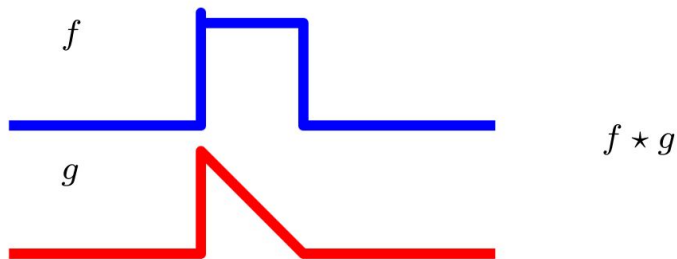


d'Alembert 1754

Convolution

Given two functions $f, g : [-\pi, \pi] \rightarrow \mathbb{R}$ their **convolution** is a function:

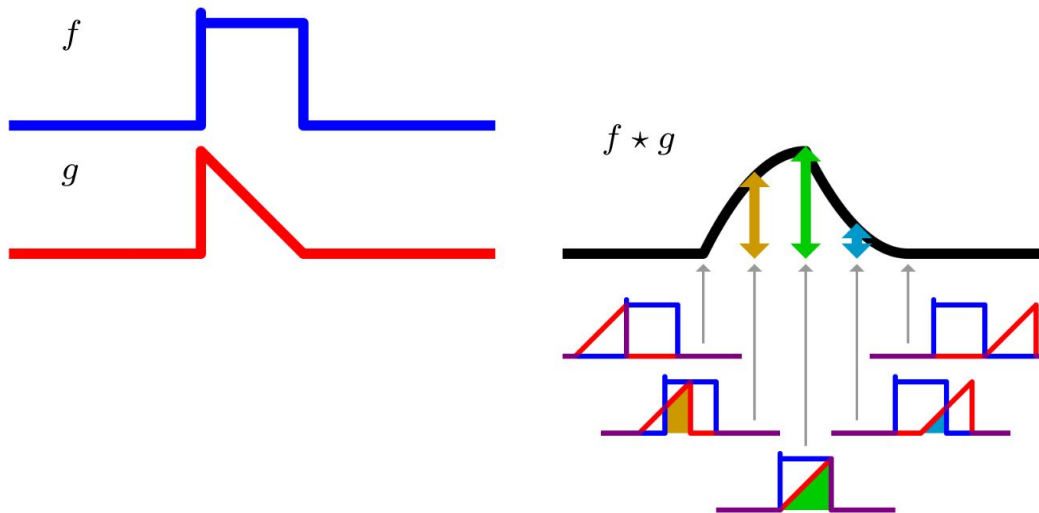
$$(f \star g)(x) = \int_{-\pi}^{\pi} f(t)g(x-t)dt$$



Convolution

Given two functions $f, g : [-\pi, \pi] \rightarrow \mathbb{R}$ their **convolution** is a function:

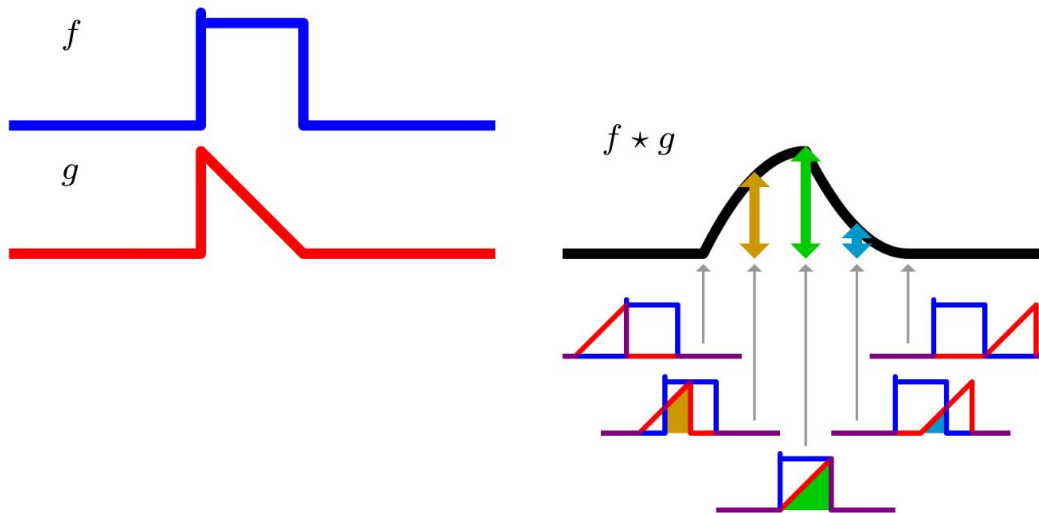
$$(f \star g)(x) = \int_{-\pi}^{\pi} f(t)g(x-t)dt$$



Convolution

Given two functions $f, g : [-\pi, \pi] \rightarrow \mathbb{R}$ their **convolution** is a function:

$$\underbrace{(f \star g)(x)}_{\text{feature map}} = \int_{-\pi}^{\pi} \underbrace{f(t)}_{\text{kernel}} \underbrace{g(x-t)}_{\text{kernel}} dt$$



Convolution: Properties

Given two functions $f, g : [-\pi, \pi] \rightarrow \mathbb{R}$ their **convolution** is a function:

$$\underbrace{(f \star g)(x)}_{\text{feature map}} = \int_{-\pi}^{\pi} f(t) \underbrace{g(x-t)}_{\text{kernel}} dt$$

Convolution is **commutative**:

$$(f \star g)(x) = \int_{-\pi}^{\pi} f(t)g(x-t)dt \stackrel{z:=x-t}{\Downarrow} \int_{-\pi}^{\pi} f(x-z)g(z)dz = (g \star f)(x)$$

Convolution: Properties

Given two functions $f, g : [-\pi, \pi] \rightarrow \mathbb{R}$ their **convolution** is a function:

$$\underbrace{(f \star g)(x)}_{\text{feature map}} = \int_{-\pi}^{\pi} f(t) \underbrace{g(x-t)}_{\text{kernel}} dt$$

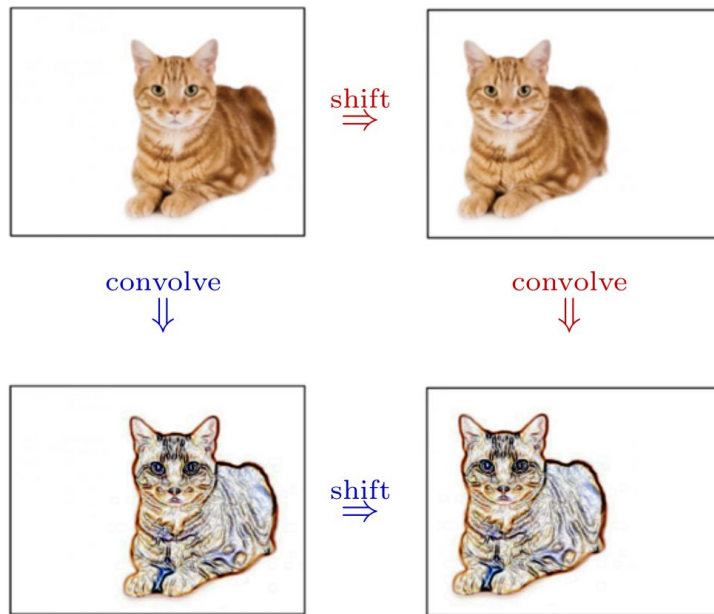
Convolution is **commutative**:

$$(f \star g)(x) = \int_{-\pi}^{\pi} f(t)g(x-t)dt \stackrel{z:=x-t}{\Downarrow} \int_{-\pi}^{\pi} f(x-z)g(z)dz = (g \star f)(x)$$

Further, convolution is **shift-equivariant**:

$$f(x - x_0) \star g(x) = (f \star g)(x - x_0)$$

Convolution: Properties



In fact, equivariance is a **defining property** of convolutions.

Convolution: Properties

We can see convolution as the application of a **linear** operator $\mathcal{G}$:

$$\mathcal{G}f(x) = (f \star g)(x) = \int_{-\pi}^{\pi} f(t) \underbrace{g(x-t)}_{\text{kernel}} dt$$

It is easy to show that $\mathcal{G}$ is linear:

$$\mathcal{G}(\alpha f(x)) = \alpha \int_{-\pi}^{\pi} f(t)g(x-t)dt = \alpha \mathcal{G}f(x)$$

Convolution: Properties

We can see convolution as the application of a **linear** operator $\mathcal{G}$:

$$\mathcal{G}f(x) = (f \star g)(x) = \int_{-\pi}^{\pi} f(t) \underbrace{g(x-t)}_{\text{kernel}} dt$$

It is easy to show that $\mathcal{G}$ is linear:

$$\begin{aligned}\mathcal{G}(\alpha f(x)) &= \alpha \int_{-\pi}^{\pi} f(t)g(x-t)dt = \alpha \mathcal{G}f(x) \\ \mathcal{G}(f+h)(x) &= \int_{-\pi}^{\pi} f(t)g(x-t)dt + \int_{-\pi}^{\pi} h(t)g(x-t)dt \\ &= \mathcal{G}f(x) + \mathcal{G}h(x)\end{aligned}$$

Convolution: Properties

We can see convolution as the application of a **linear** operator $\mathcal{G}$:

$$\mathcal{G}f(x) = (f \star g)(x) = \int_{-\pi}^{\pi} f(t) \underbrace{g(x-t)}_{\text{kernel}} dt$$

It is easy to show that $\mathcal{G}$ is linear:

$$\begin{aligned}\mathcal{G}(\alpha f(x)) &= \alpha \int_{-\pi}^{\pi} f(t)g(x-t)dt = \alpha \mathcal{G}f(x) \\ \mathcal{G}(f+h)(x) &= \int_{-\pi}^{\pi} f(t)g(x-t)dt + \int_{-\pi}^{\pi} h(t)g(x-t)dt \\ &= \mathcal{G}f(x) + \mathcal{G}h(x)\end{aligned}$$

Translation **equivariance** can then be phrased as:

$$\mathcal{G}(\mathcal{T}f) = \mathcal{T}(\mathcal{G}f)$$

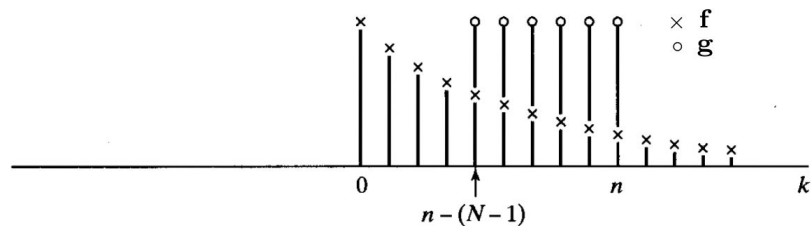
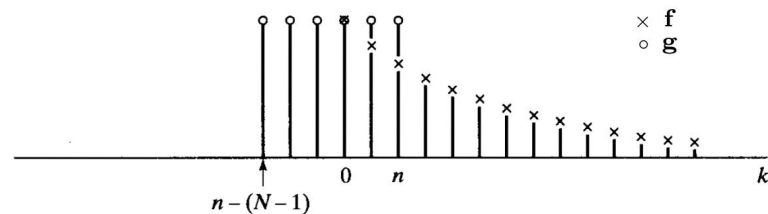
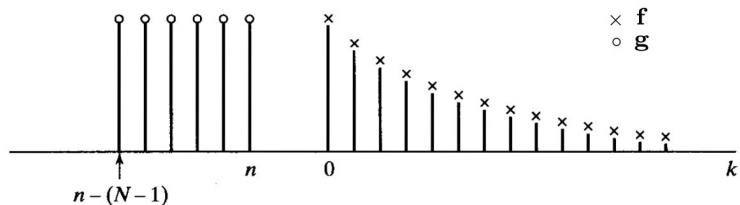
i.e., the convolution and translation operators **commute**.

Discrete Convolutions

In the **discrete** setting, we deal with vectors $\mathbf{f}, \mathbf{g}$.

We define the **convolution sum**:

$$(\mathbf{f} \star \mathbf{g})[n] = \sum_{k=-\infty}^{\infty} \mathbf{f}[k] \mathbf{g}[n - k]$$

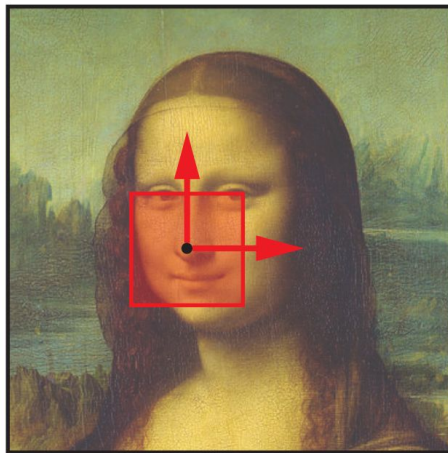


Discrete Convolutions

On 2D domains (e.g. RGB images $f : \mathbb{R}^2 \rightarrow \mathbb{R}^3$), for each channel:

$$(\mathbf{f} \star \mathbf{g})[m, n] = \sum_k \sum_\ell \mathbf{f}[k, \ell] \mathbf{g}[m - k, n - \ell]$$

We get the classical interpretation in terms of a moving window:

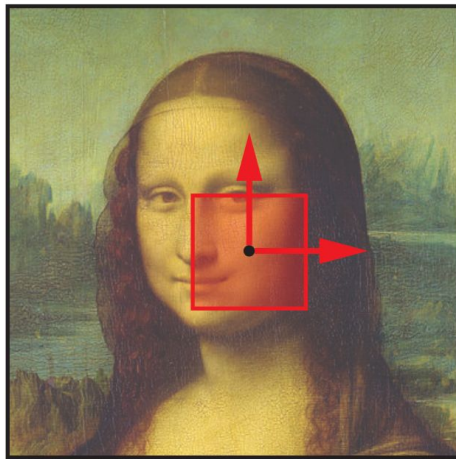


Discrete Convolutions

On 2D domains (e.g. RGB images $f : \mathbb{R}^2 \rightarrow \mathbb{R}^3$), for each channel:

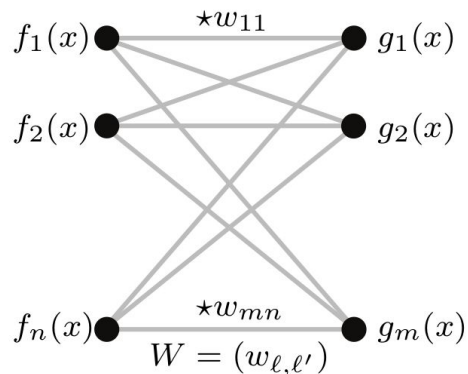
$$(\mathbf{f} \star \mathbf{g})[m, n] = \sum_k \sum_\ell \mathbf{f}[k, \ell] \mathbf{g}[m - k, n - \ell]$$

We get the classical interpretation in terms of a moving window:



Convolutional Neural Network

Main idea: Compose equivariant layers implemented via convolution.



Single convolutional layer

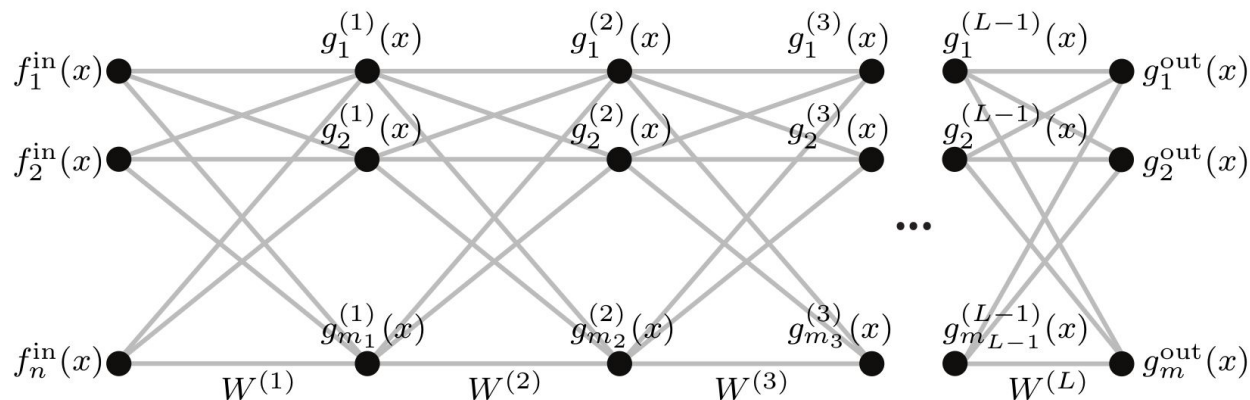
Conv. layer
$$g_{\ell}(x) = \sigma \left(\sum_{\ell'=1}^n (f_{\ell'} \star w_{\ell, \ell'})(x) \right) \quad \begin{array}{l} \ell = 1, \dots, m \\ \ell' = 1, \dots, n \end{array}$$

Activation, e.g. $\sigma(x) = \max\{x, 0\}$ rectified linear unit (ReLU)

Parameters filters W

Convolutional Neural Network

Main idea: Compose equivariant layers implemented via convolution.



CNN consisting of L convolutional layers

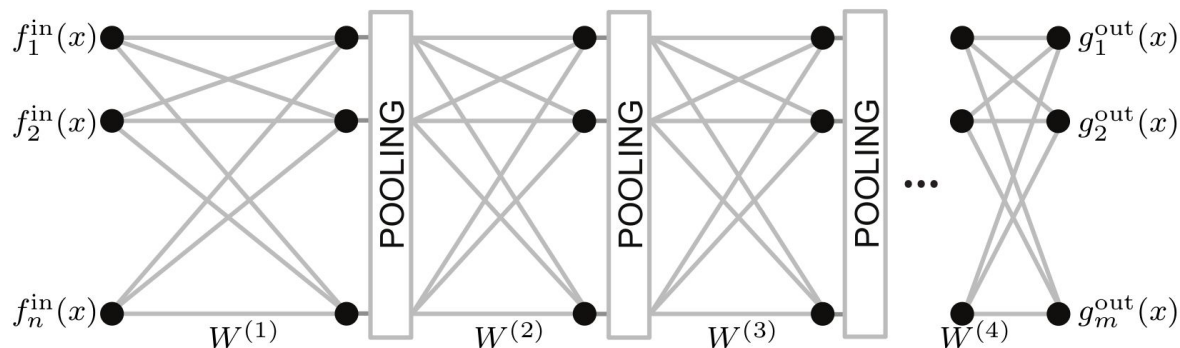
Conv. layer
$$g_{\ell}^{(k)}(x) = \sigma \left(\sum_{\ell'=1}^{m_{k-1}} (g_{\ell'}^{(k-1)} \star w_{\ell, \ell'}^{(k)})(x) \right) \quad \begin{array}{l} \ell = 1, \dots, m_k \\ \ell' = 1, \dots, m_{k-1} \end{array}$$

Activation, e.g. $\sigma(x) = \max\{x, 0\}$ rectified linear unit (ReLU)

Parameters filters of all layers $W^{(1)}, \dots, W^{(L)}$

Convolutional Neural Network

Main idea: Compose equivariant layers implemented via convolution.



CNN consisting of L convolutional layers interleaved with pooling

Conv. layer
$$g_\ell^{(k)}(x) = \sigma \left(\sum_{\ell'=1}^{m_{k-1}} (g_{\ell'}^{(k-1)} \star w_{\ell, \ell'}^{(k)})(x) \right) \quad \begin{array}{l} \ell = 1, \dots, m_k \\ \ell' = 1, \dots, m_{k-1} \end{array}$$

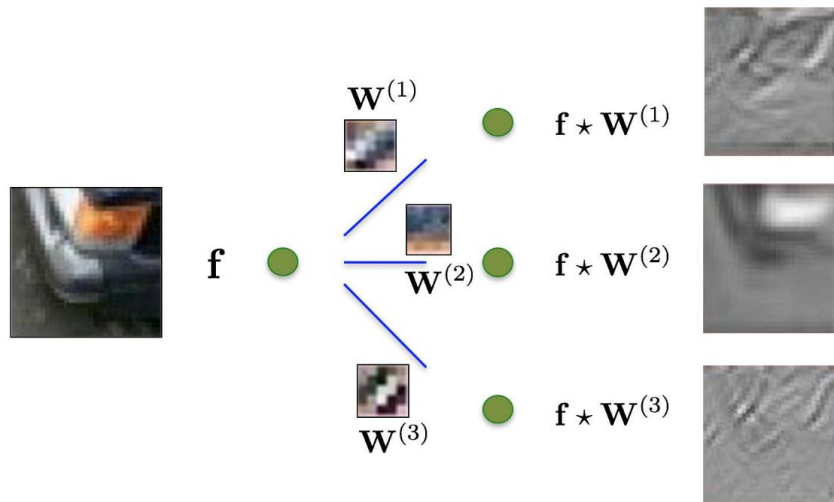
Activation, e.g. $\sigma(x) = \max\{x, 0\}$ rectified linear unit (ReLU)

Parameters filters of all layers $W^{(1)}, \dots, W^{(L)}$

Pooling
$$g_\ell^{(k)}(x) = \|g_\ell^{(k-1)}(x') : x' \in \mathcal{N}(x)\|_n \quad n = 1, 2, \text{ or } \infty$$

Local Filters

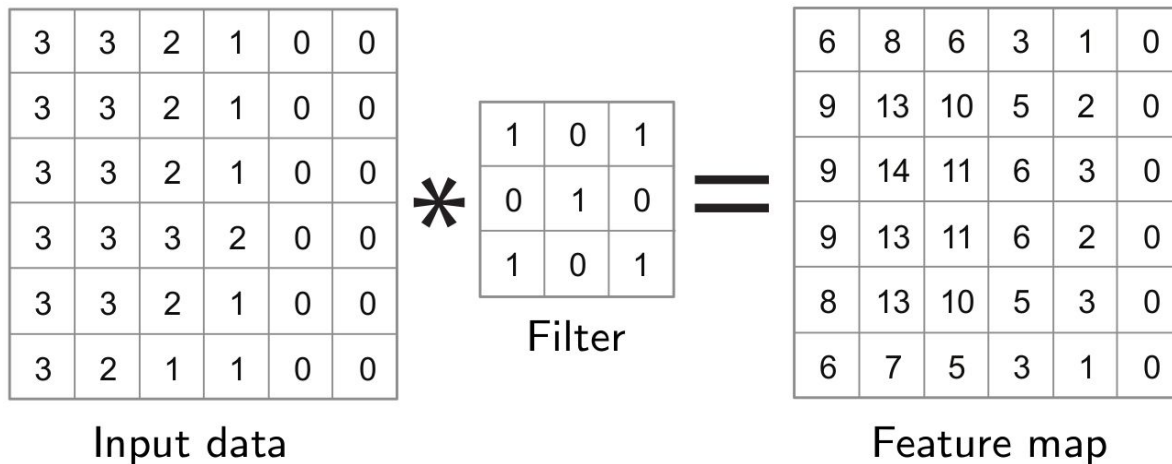
Shift-invariance is implemented via **convolutional operators**.



- $O(1)$ parameters per filter; huge gain compared to the MLP.
- Filter weights are applied across the entire image $\Rightarrow$ **weight sharing**.

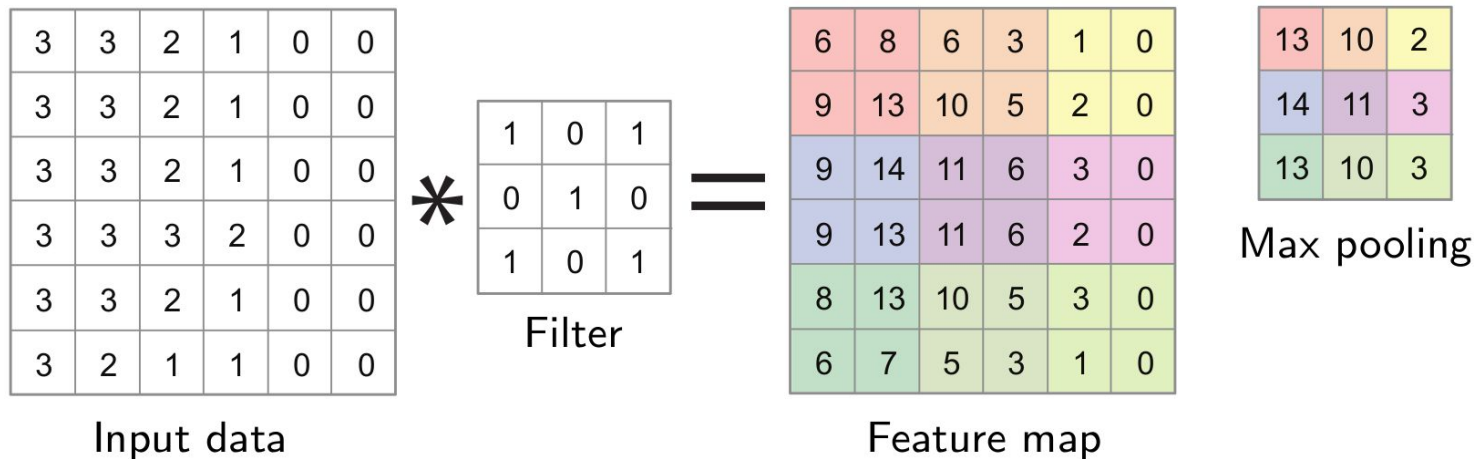
Pooling

At deep layers, filters interact with larger portions of the input.



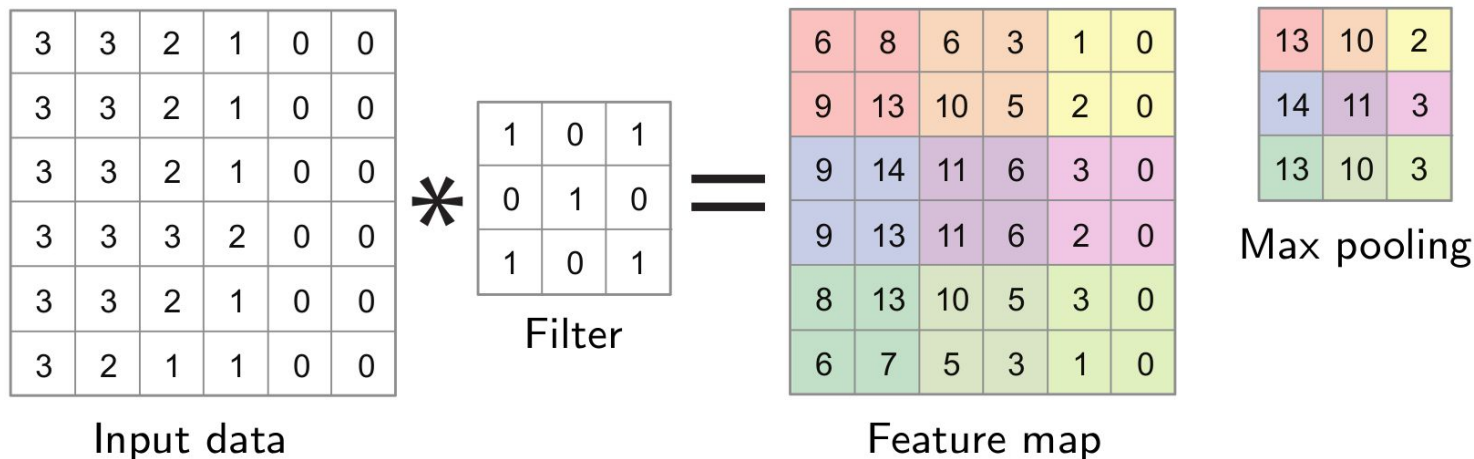
Pooling

At deep layers, filters interact with larger portions of the input.



Pooling

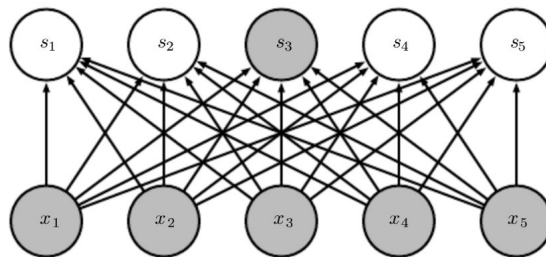
At deep layers, filters interact with larger portions of the input.



This allows to capture complicated **non-local interactions** via simple building blocks that only describe sparse interactions.

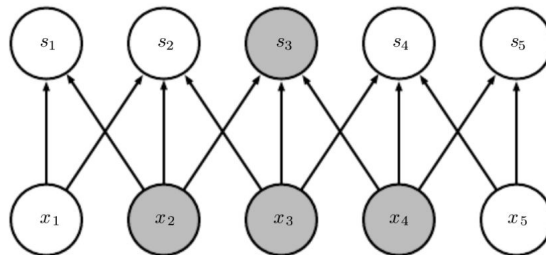
Sparse Interaction

Fully-connected layer:



each edge is a different weight

Convolutional layer:



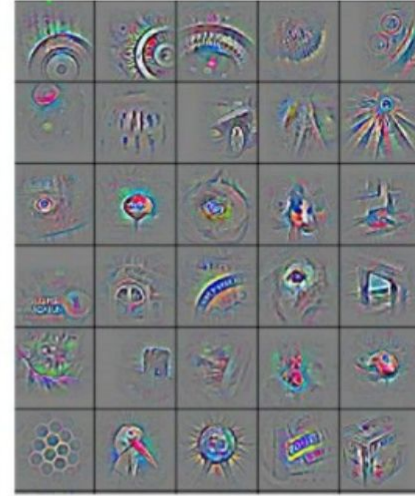
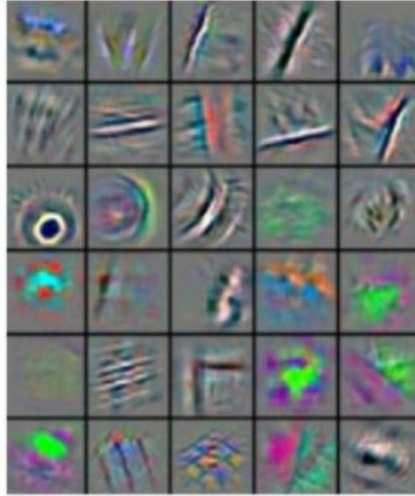
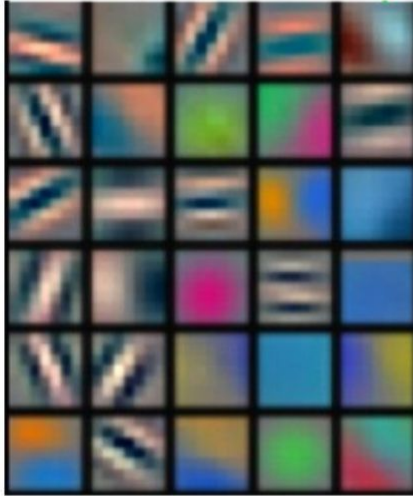
the outgoing edges have the same weights
for each input variable (**weight sharing**)

Image: Goodfellow et al, 2016

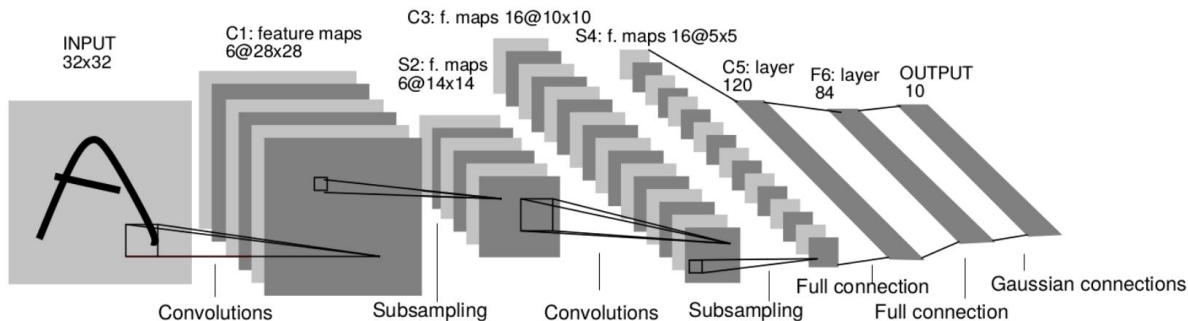
Learned Features



→ low-level features → mid-level features → hi-level features → “car”



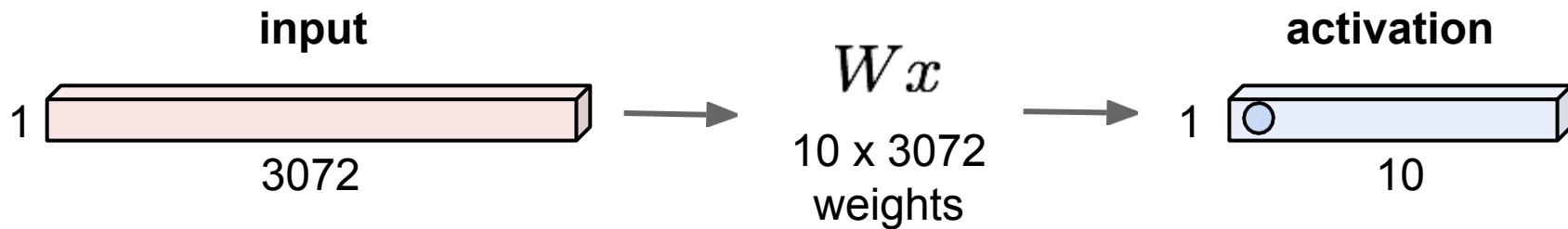
Key Properties of CNN



- Convolutional filters (**Translation equivariance**)
- Multiple layers (**Compositionality**)
- Filters localized in space (**Locality**)
- Weight sharing (**Self-similarity**)
- $\mathcal{O}(1)$ parameters per filter (independent of input image size n)

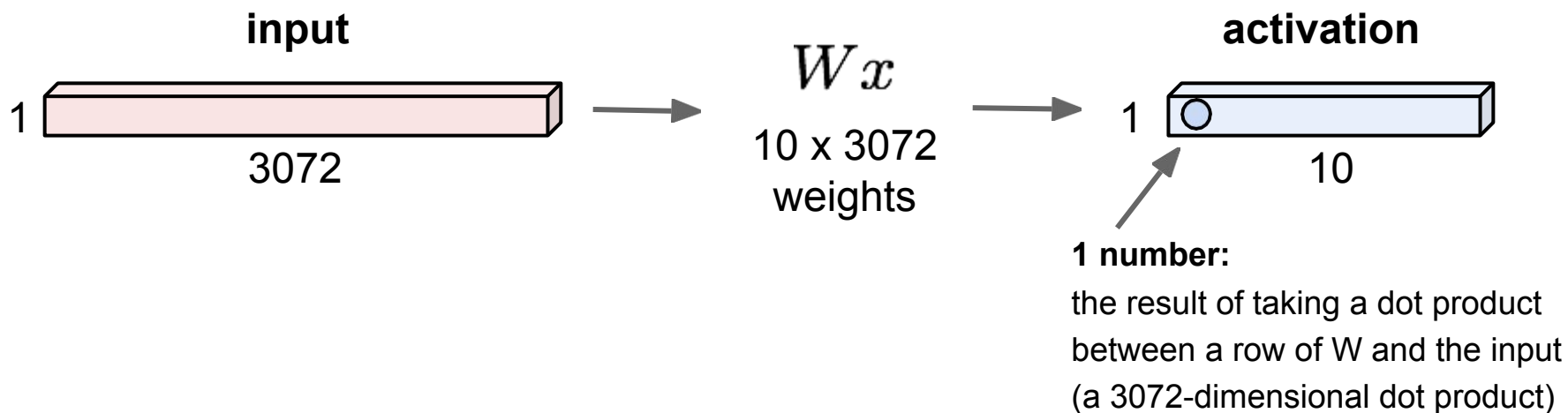
Fully Connected Layer

32x32x3 image -> stretch to 3072 x 1



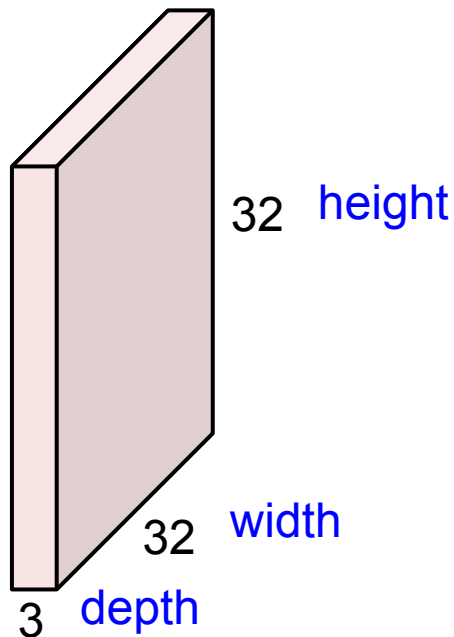
Fully Connected Layer

32x32x3 image -> stretch to 3072 x 1



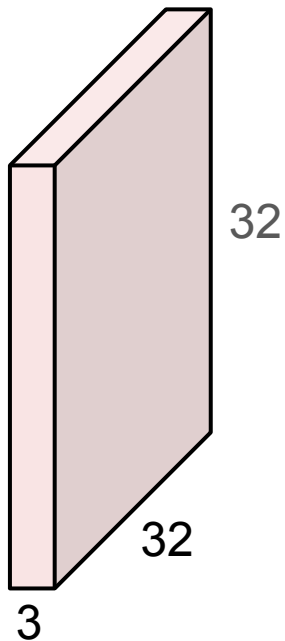
Convolution Layer

32x32x3 image -> preserve spatial structure



Convolution Layer

32x32x3 image



5x5x3 filter

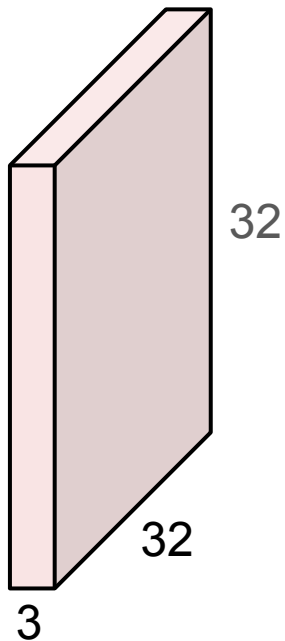


Convolve the filter with the image

i.e. “slide over the image spatially,
computing dot products”

Convolution Layer

32x32x3 image



Filters always extend the full depth of the input volume

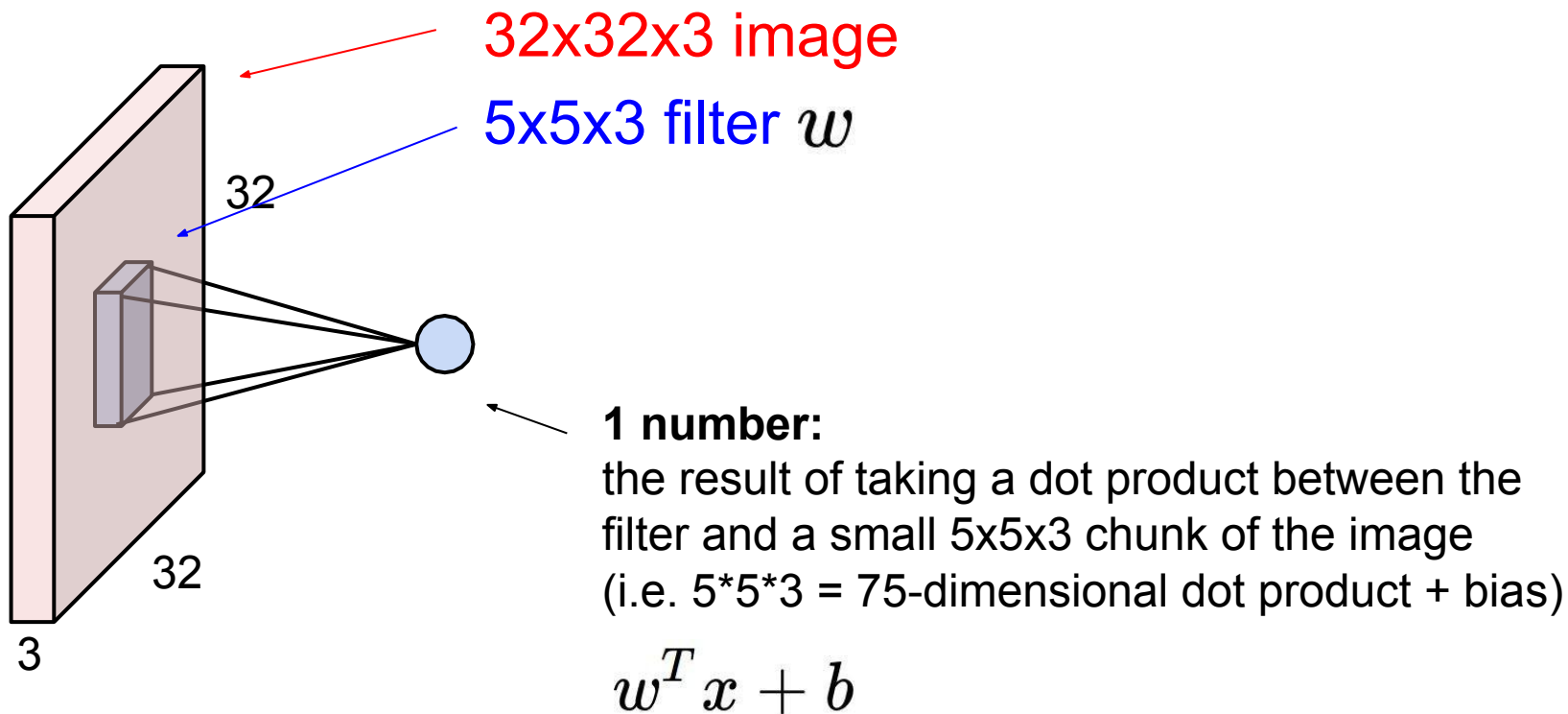
5x5x3 filter



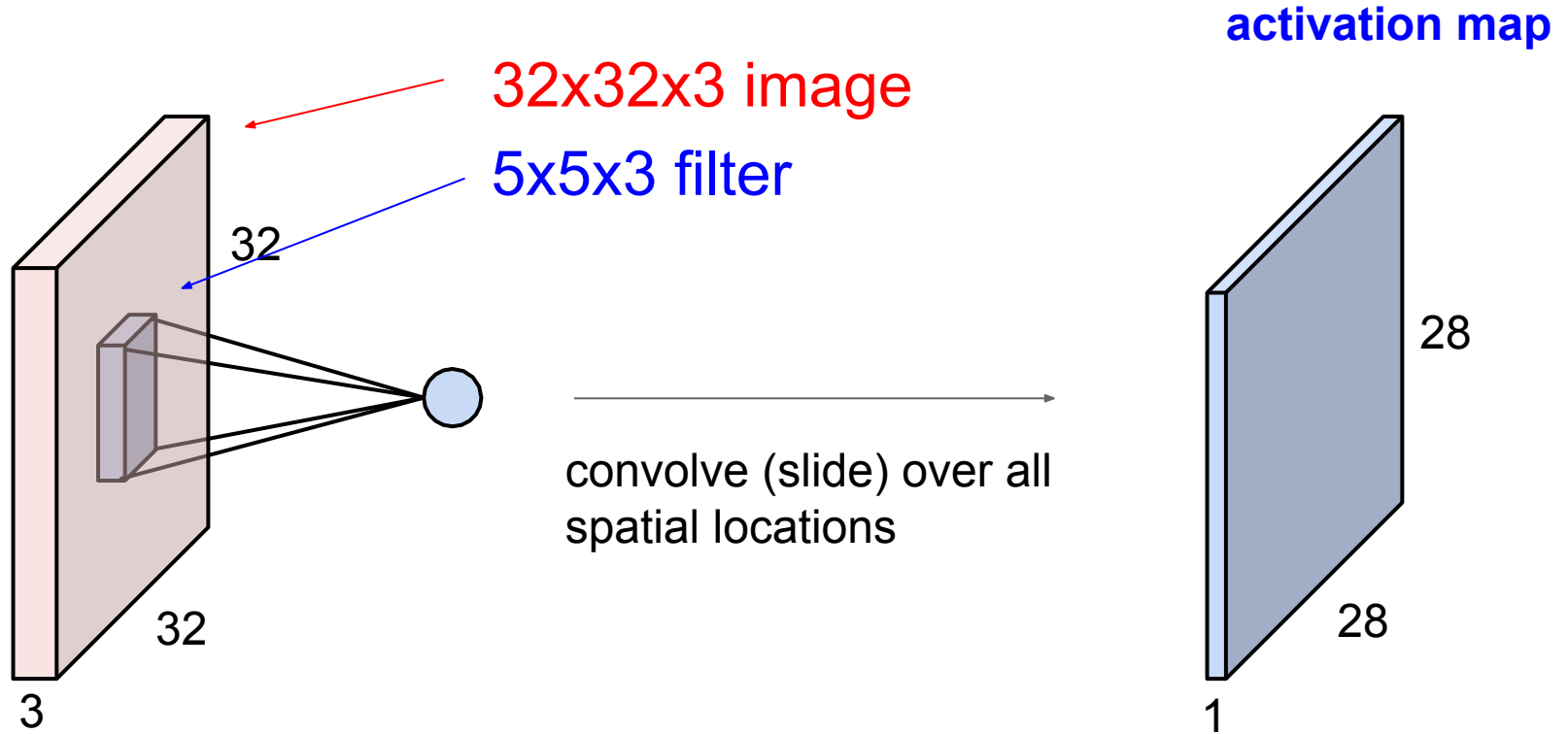
Convolve the filter with the image

i.e. “slide over the image spatially, computing dot products”

Convolution Layer

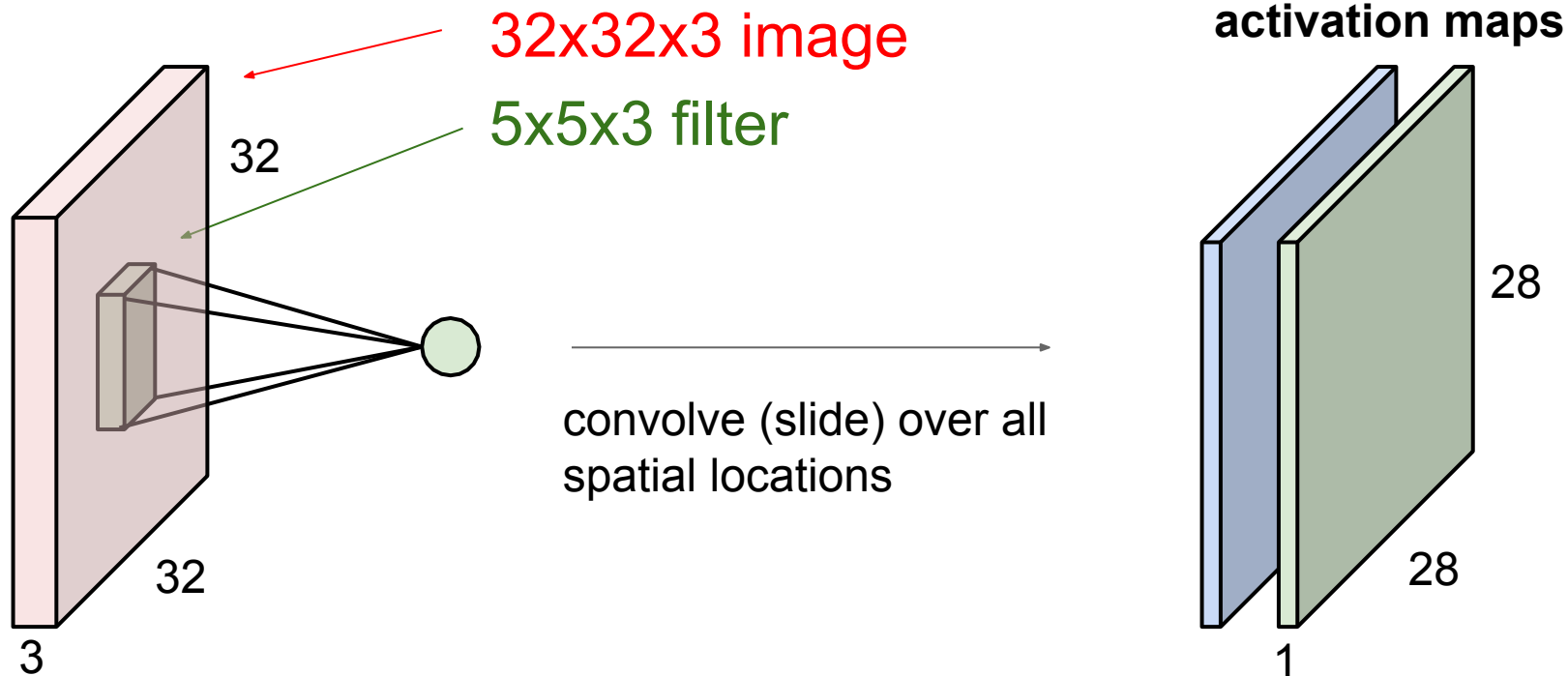


Convolution Layer

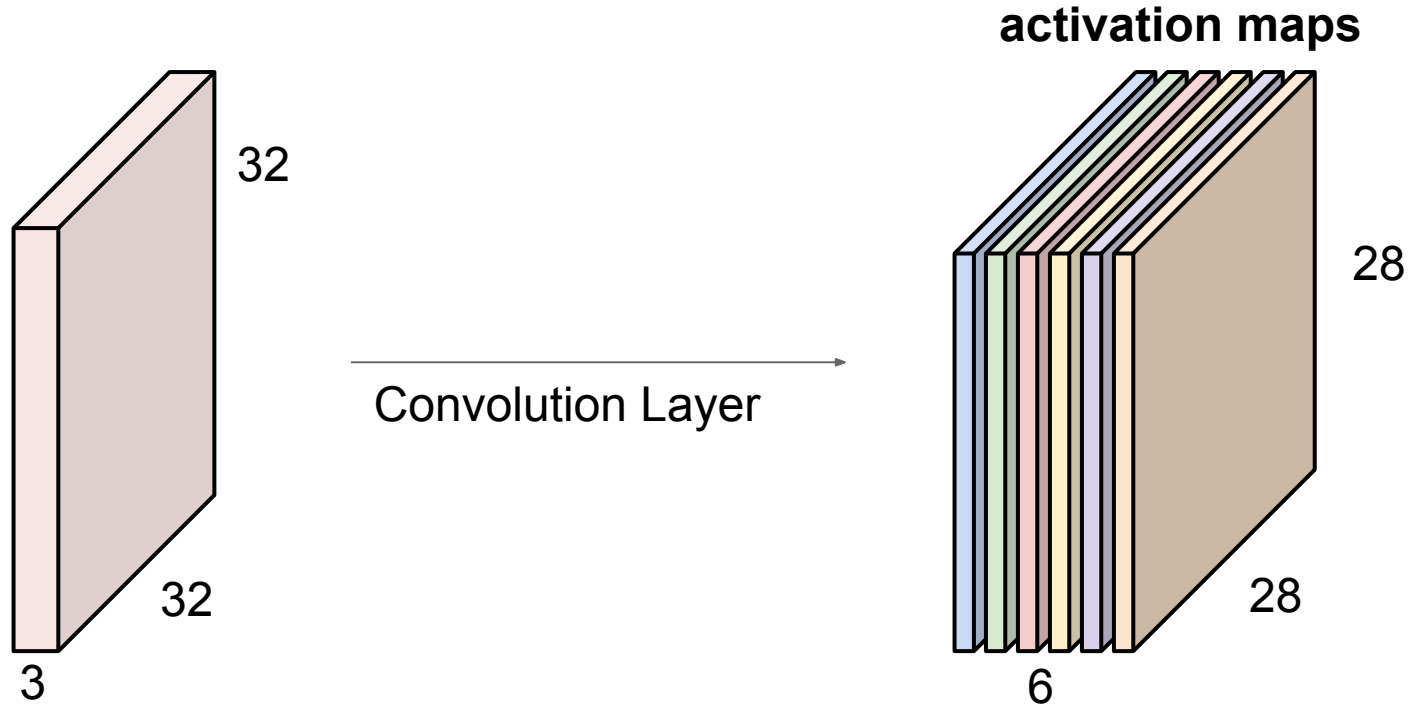


Convolution Layer

consider a second, **green** filter

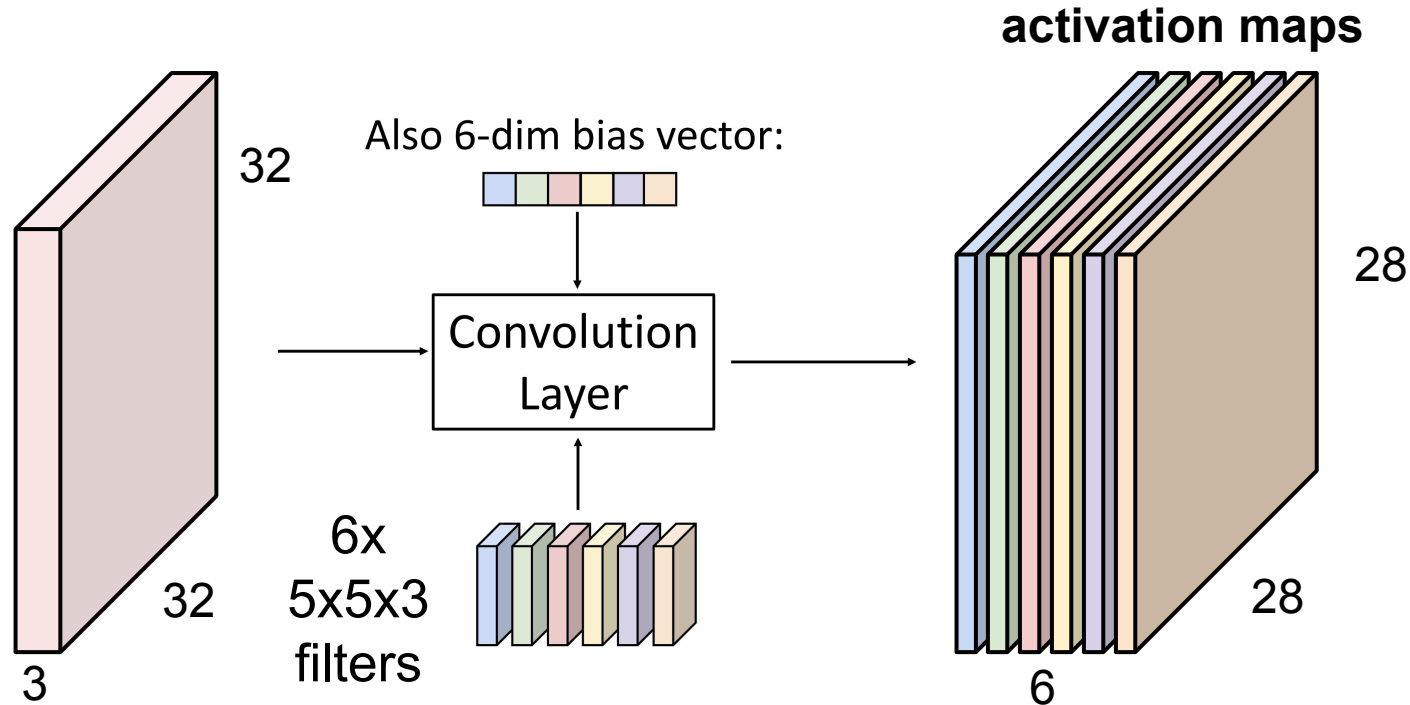


For example, if we had 6 5x5 filters, we'll get 6 separate activation maps:



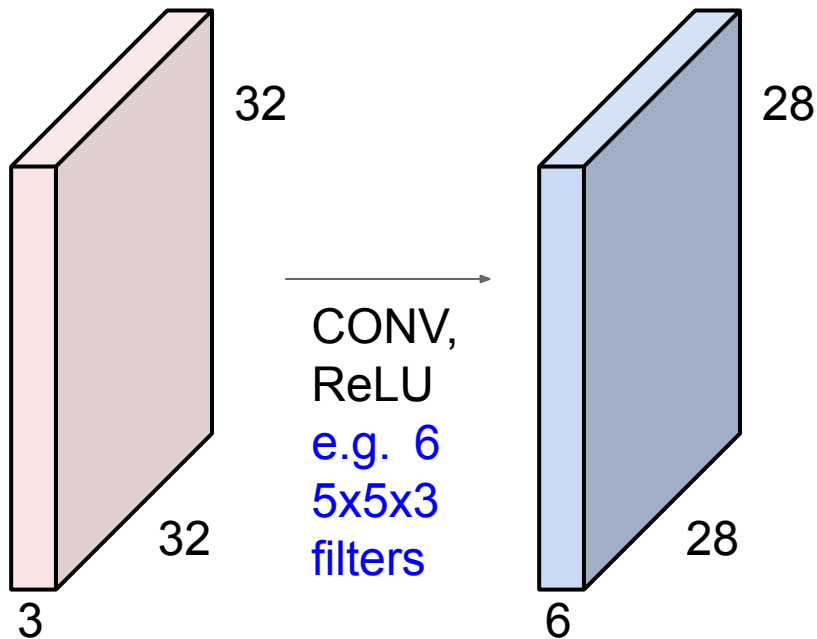
We stack these up to get a “new image” of size 28x28x6!

For example, if we had 6 5x5 filters, we'll get 6 separate activation maps:

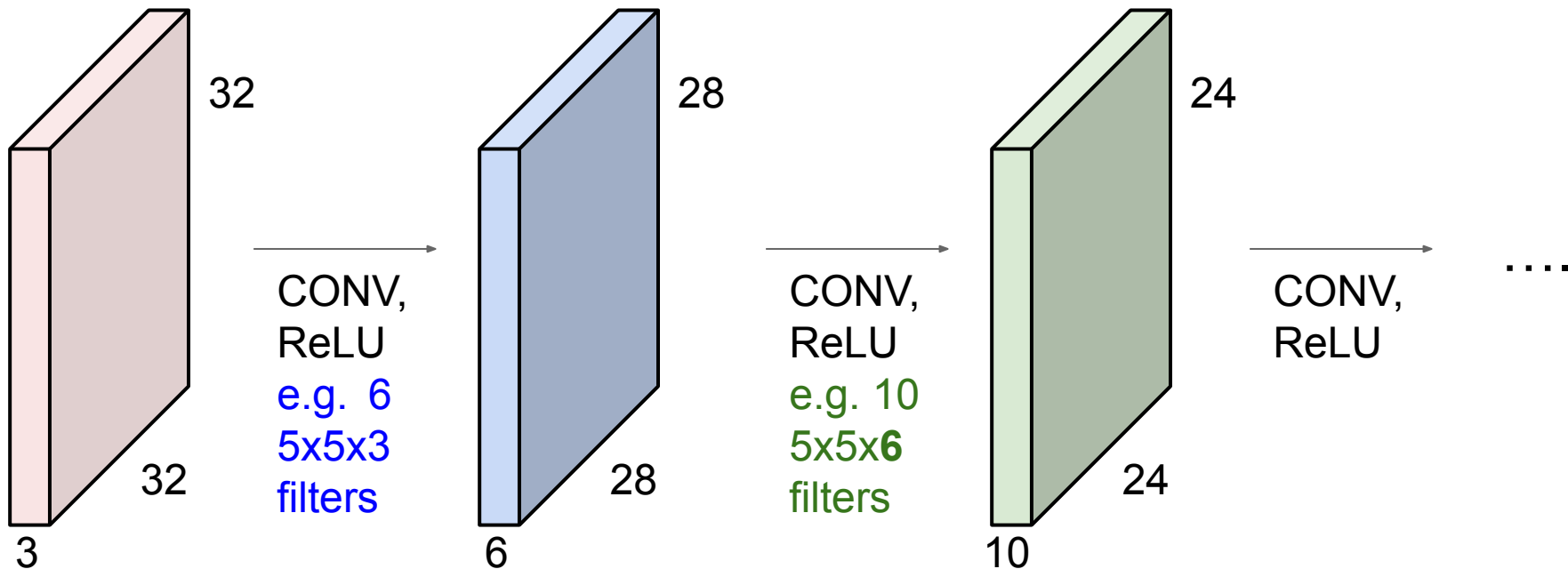


We stack these up to get a “new image” of size 28x28x6!

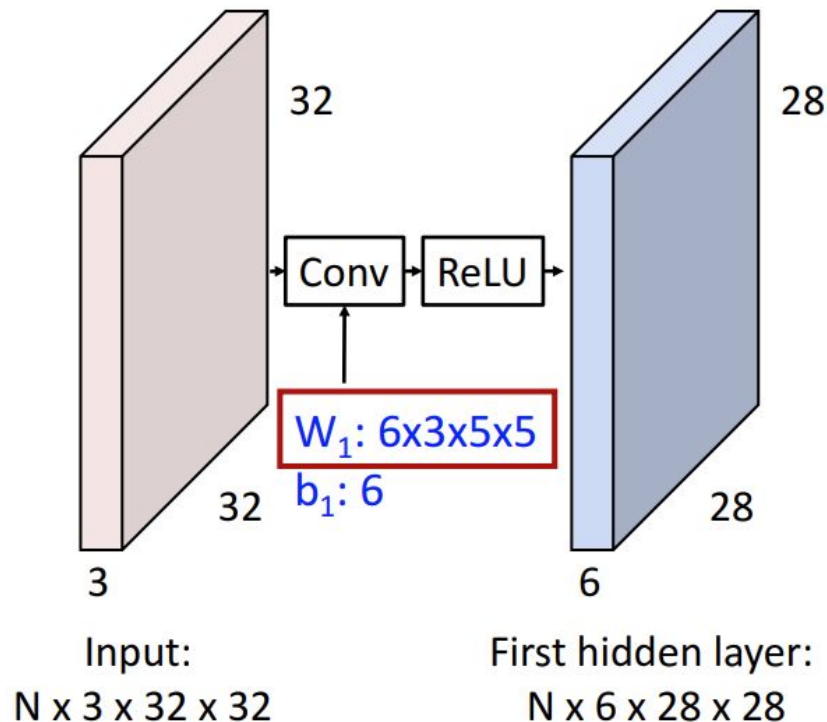
Preview: ConvNet is a sequence of Convolution Layers, interspersed with activation functions



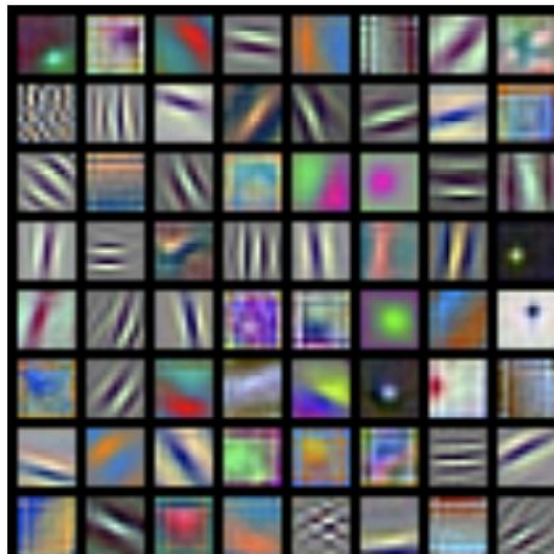
Preview: ConvNet is a sequence of Convolutional Layers, interspersed with activation functions



What do convolutional filters learn?

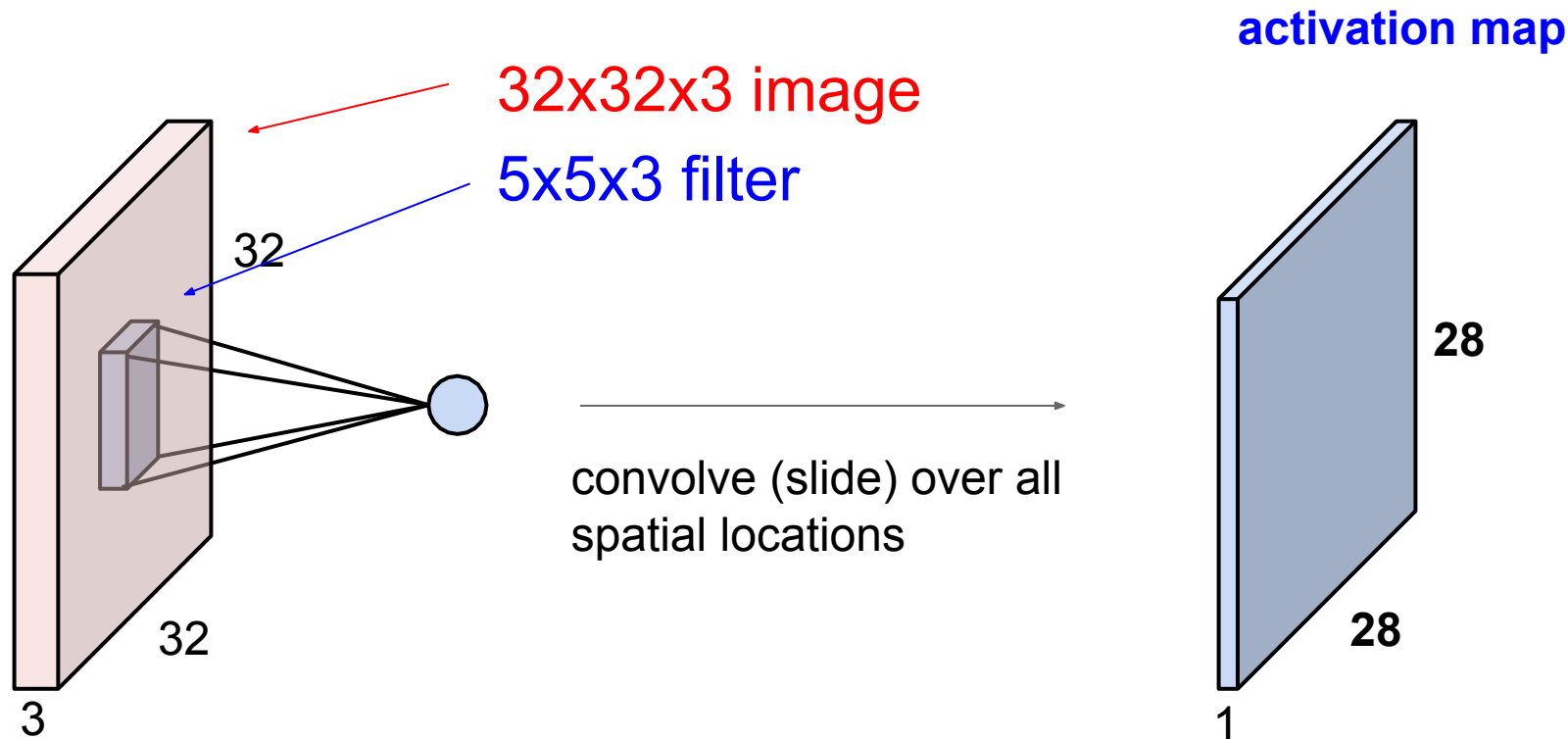


First-layer conv filters: local image templates
(Often learns oriented edges, opposing colors)



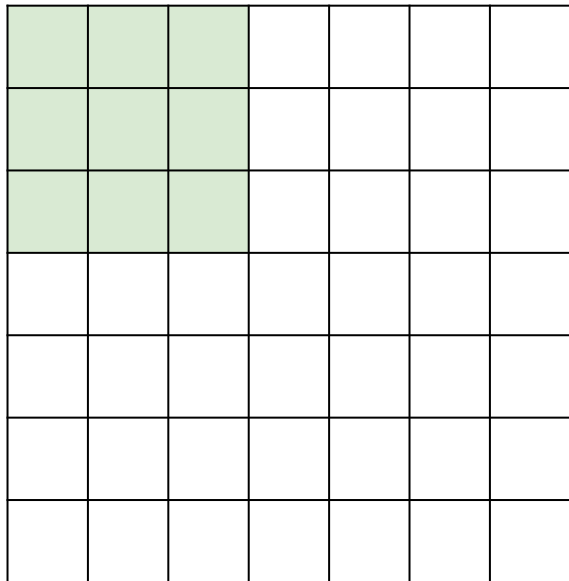
AlexNet: 64 filters, each $3 \times 11 \times 11$

A closer look at spatial dimensions:



A closer look at spatial dimensions:

7

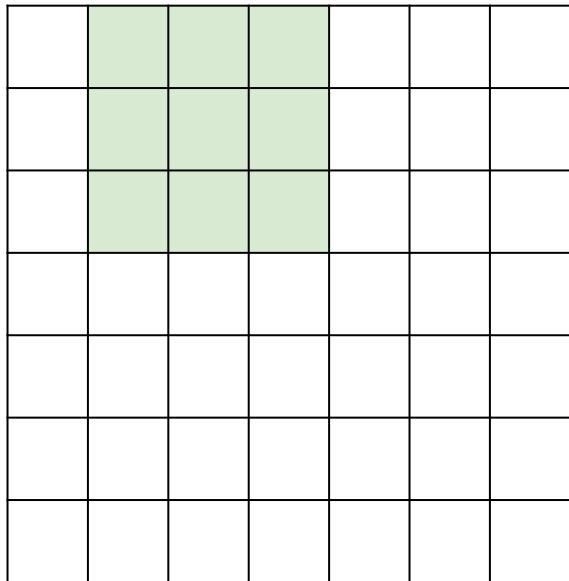


7

7x7 input (spatially)
assume 3x3 filter

A closer look at spatial dimensions:

7

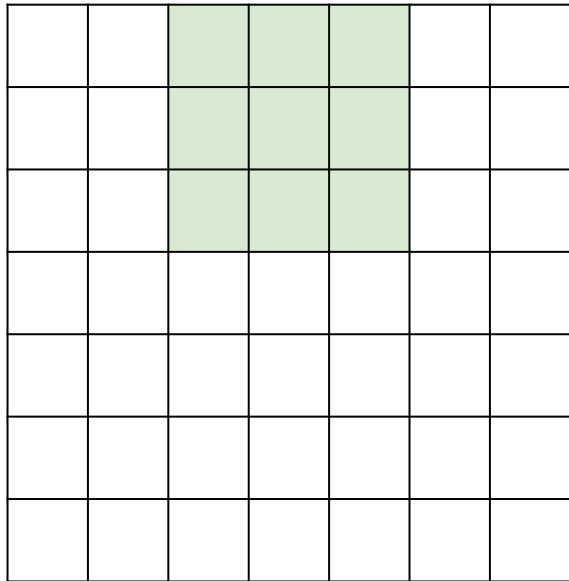


7

7x7 input (spatially)
assume 3x3 filter

A closer look at spatial dimensions:

7

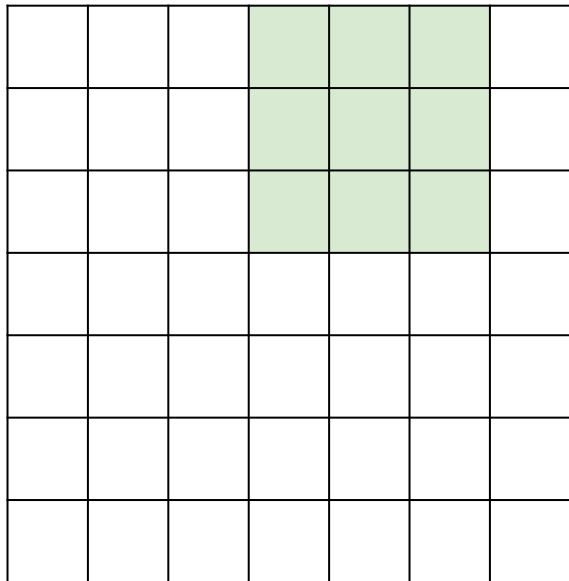


7

7x7 input (spatially)
assume 3x3 filter

A closer look at spatial dimensions:

7



7

7x7 input (spatially)
assume 3x3 filter

A closer look at spatial dimensions:

7

7

7x7 input (spatially)
assume 3x3 filter

=> 5x5 output

A closer look at spatial dimensions:

N

			F			
					F	

7x7 input (spatially)
assume 3x3 filter

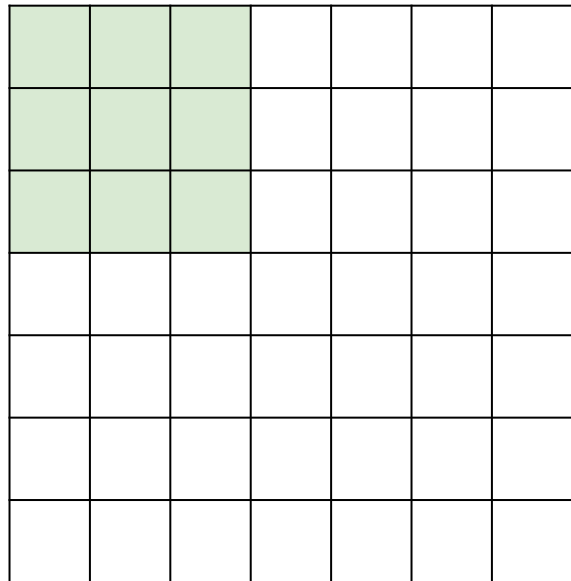
N $\Rightarrow$ 5x5 output

Output size:
 $(N - F) / \text{stride} + 1$

Problem: Feature
maps “shrink”
with each layer!

A closer look at spatial dimensions:

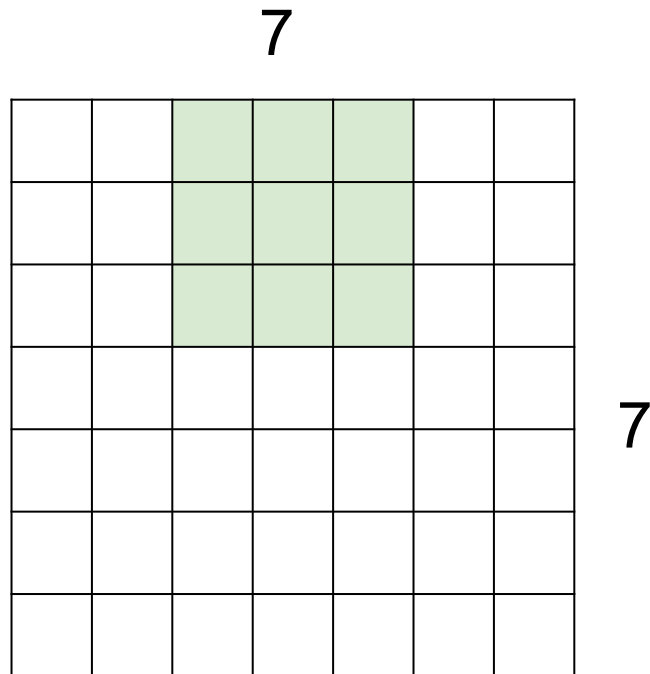
7



7

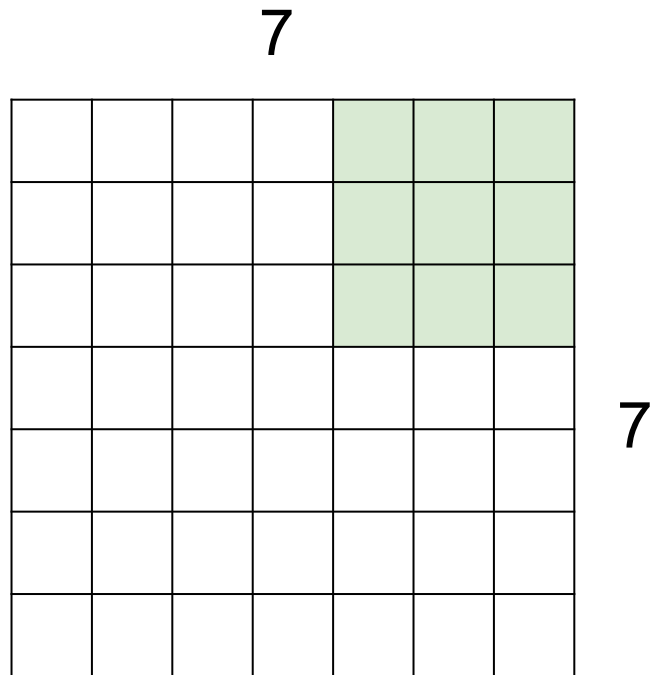
7x7 input (spatially)
assume 3x3 filter
applied **with stride 2**

A closer look at spatial dimensions:



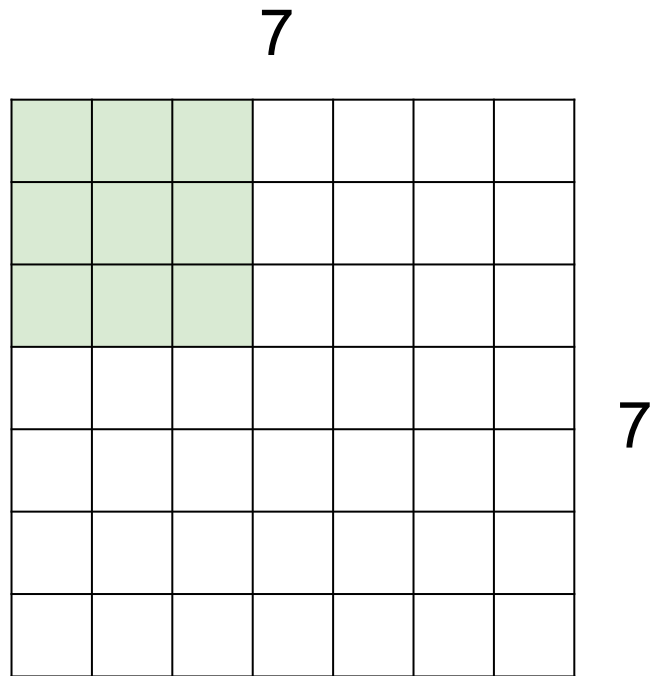
7x7 input (spatially)
assume 3x3 filter
applied **with stride 2**

A closer look at spatial dimensions:



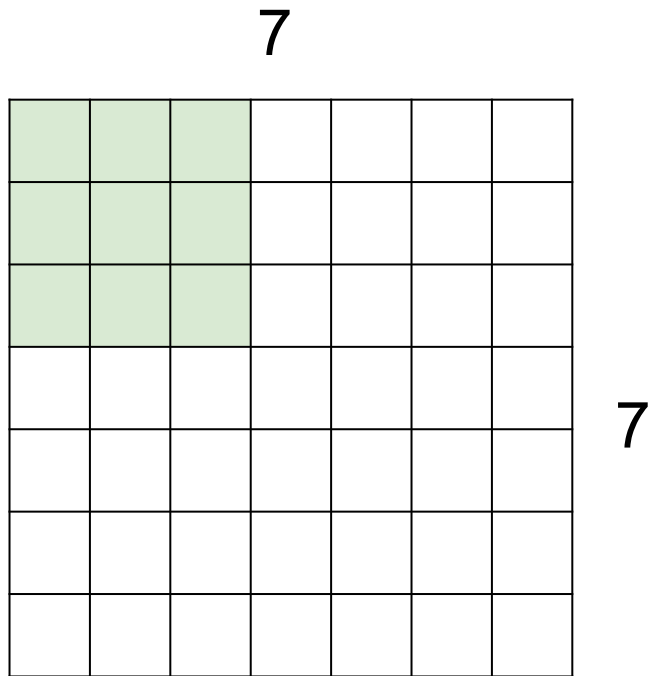
7x7 input (spatially)
assume 3x3 filter
applied **with stride 2**
=> 3x3 output!

A closer look at spatial dimensions:



7x7 input (spatially)
assume 3x3 filter
applied **with stride 3?**

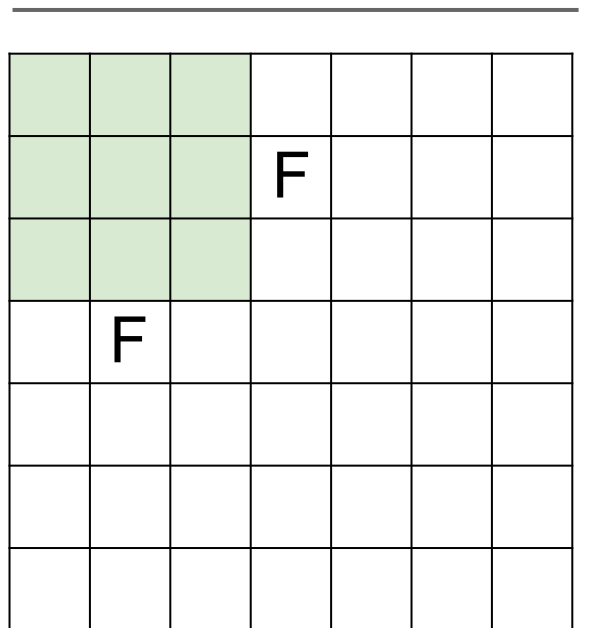
A closer look at spatial dimensions:



7x7 input (spatially)
assume 3x3 filter
applied **with stride 3?**

doesn't fit!
cannot apply 3x3 filter on
7x7 input with stride 3.

N



Output size:

$$(N - F) / \text{stride} + 1$$

e.g. $N = 7$, $F = 3$:

$$\text{stride } 1 \Rightarrow (7 - 3) / 1 + 1 = 5$$

$$\text{stride } 2 \Rightarrow (7 - 3) / 2 + 1 = 3$$

$$\text{stride } 3 \Rightarrow (7 - 3) / 3 + 1 = 2.33 \therefore \backslash$$

In practice: Common to zero pad the border

0	0	0	0	0	0			
0								
0								
0								
0								

e.g. input 7x7

3x3 filter, applied with **stride 1**

pad with 1 pixel border => what is the output?

(recall:)

$$(N - F) / \text{stride} + 1$$

In practice: Common to zero pad the border

0	0	0	0	0	0			
0								
0								
0								
0								

e.g. input 7x7

3x3 filter, applied with **stride 1**

pad with 1 pixel border => what is the output?

7x7 output!

In practice: Common to zero pad the border

0	0	0	0	0	0			
0								
0								
0								
0								

e.g. input 7x7

3x3 filter, applied with **stride 1**

pad with 1 pixel border => what is the output?

7x7 output!

in general, common to see CONV layers with stride 1, filters of size $F \times F$, and zero-padding with $(F-1)/2$. (will preserve size spatially)

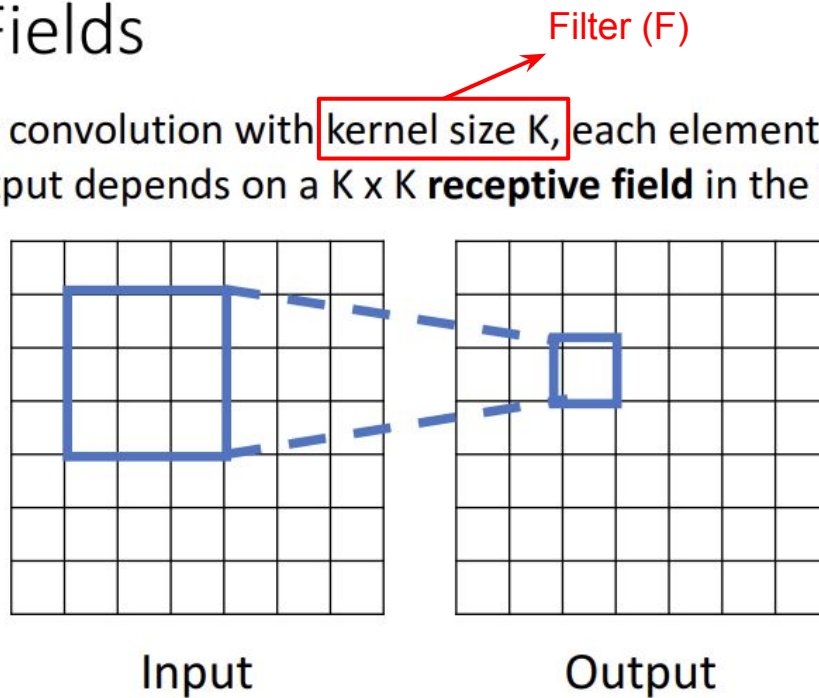
e.g. $F = 3 \Rightarrow$ zero pad with 1

$F = 5 \Rightarrow$ zero pad with 2

$F = 7 \Rightarrow$ zero pad with 3

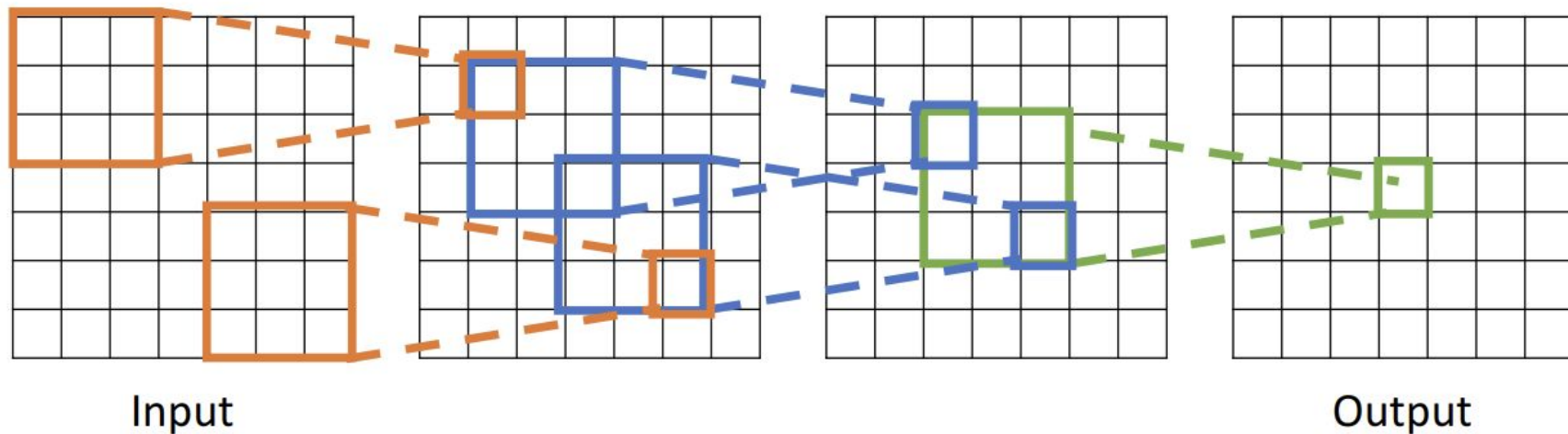
Receptive Fields

For convolution with **kernel size K** , each element in the output depends on a $K \times K$ **receptive field** in the input



Receptive Fields

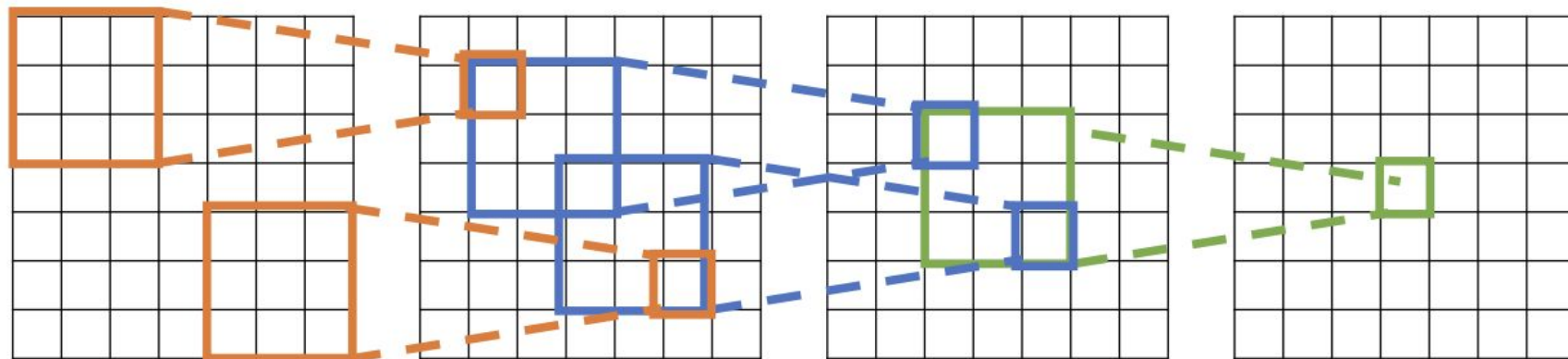
Each successive convolution adds $K - 1$ to the receptive field size
With L layers the receptive field size is $1 + L * (K - 1)$



Be careful – “receptive field in the input” vs “receptive field in the previous layer”
Hopefully clear from context!

Receptive Fields

Each successive convolution adds $K - 1$ to the receptive field size
With L layers the receptive field size is $1 + L * (K - 1)$



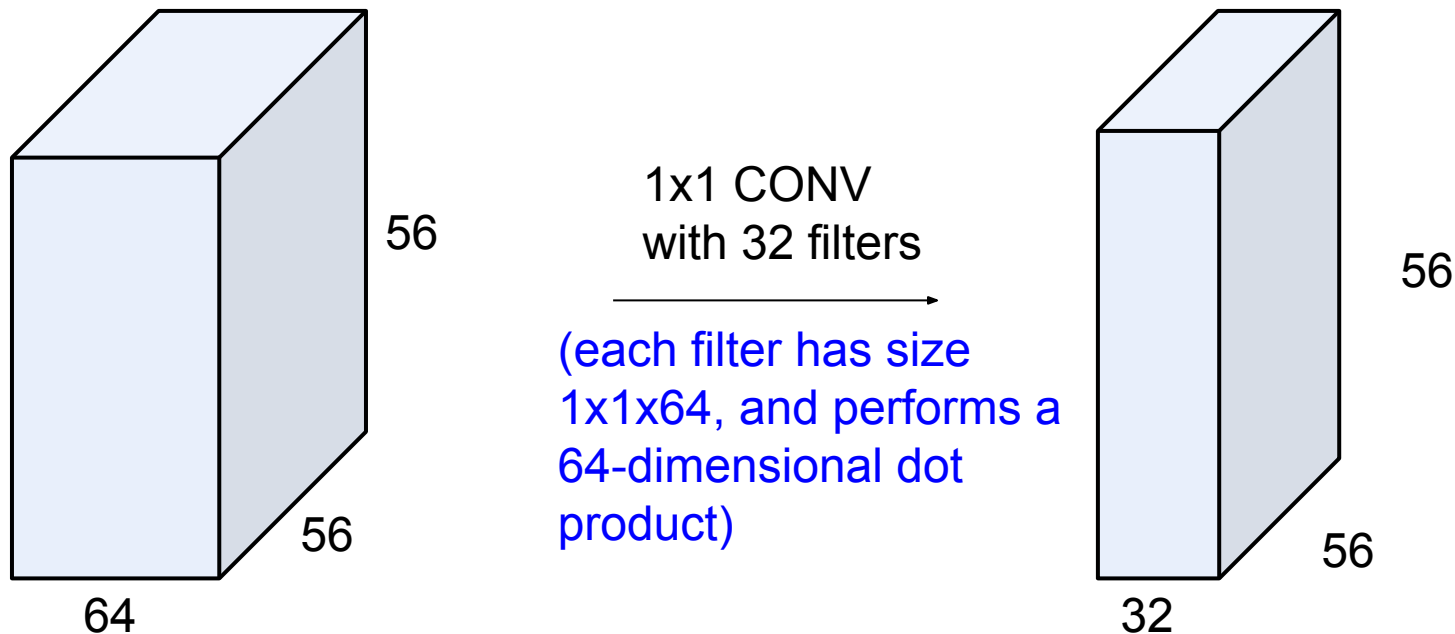
Input

Problem: For large images we need many layers
for each output to “see” the whole image

Solution: Downsample inside the network

Output

(btw, 1x1 convolution layers make perfect sense)

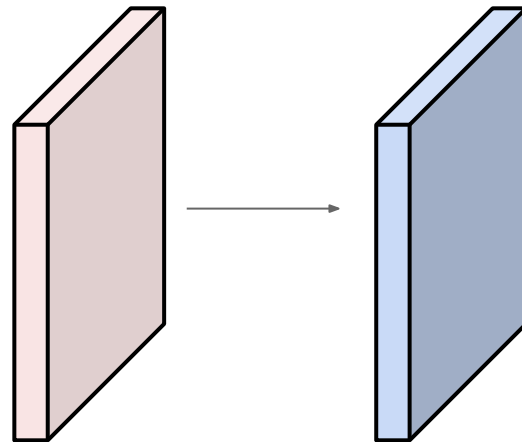


Examples time:

Input volume: **32x32x3**

10 5x5 filters with stride 1, pad 2

Output volume size: ?



Examples time:

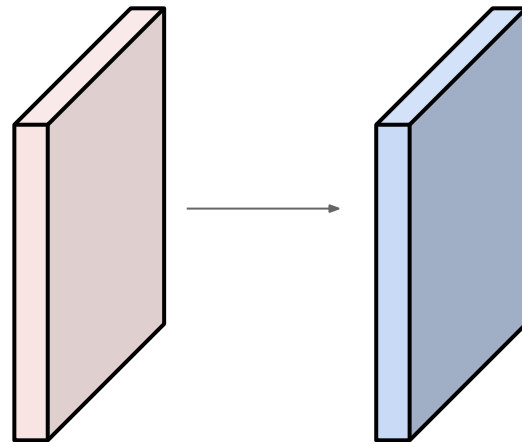
Input volume: **32x32x3**

10 **5x5** filters with stride **1**, pad **2**

Output volume size:

$(32 + 2 * 2 - 5) / 1 + 1 = 32$ spatially, so

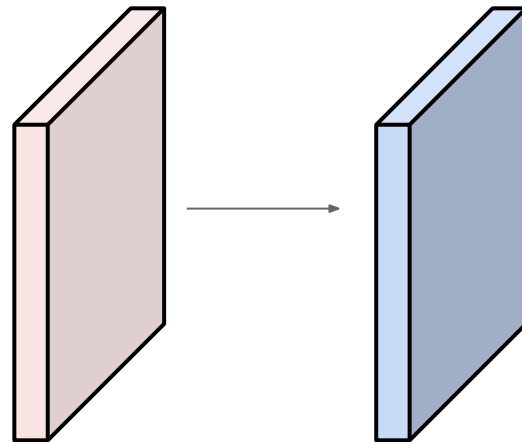
32x32x10



Examples time:

Input volume: **32x32x3**

10 5x5 filters with stride 1, pad 2

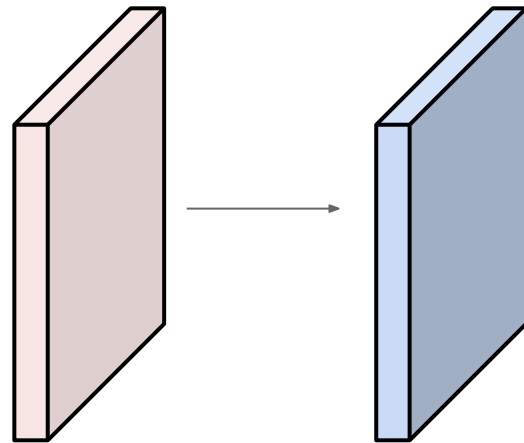


Number of parameters in this layer?

Examples time:

Input volume: **32x32x3**

10 **5x5** filters with stride 1, pad 2



Number of parameters in this layer?

each filter has $5*5*3 + 1 = 76$ params

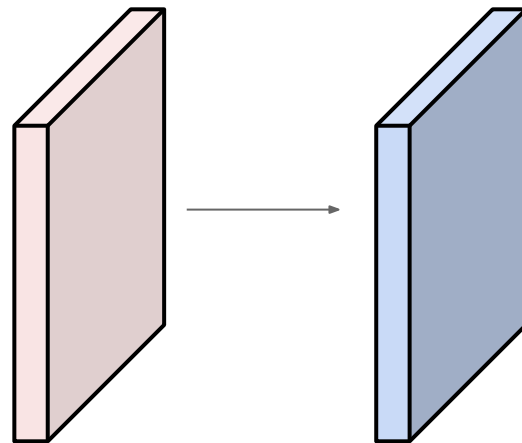
(+1 for bias)

$\Rightarrow 76*10 = 760$

Examples time:

Input volume: **32x32x3**

10 **5x5** filters with stride 1, pad 2



Number of parameters in this layer?

$$\Rightarrow 76 * 10 = 760$$

(+1 for bias)

Number of multiply-add operations?

$10 * 32 * 32 = 10240$ outputs; each output is the inner product of two $5 \times 5 \times 3$ tensors (75 elems); total = $75 * 10240 = 768k$

Summary. To summarize, the Conv Layer:

- Accepts a volume of size $W_1 \times H_1 \times D_1$
- Requires four hyperparameters:
 - Number of filters K ,
 - their spatial extent F ,
 - the stride S ,
 - the amount of zero padding P .
- Produces a volume of size $W_2 \times H_2 \times D_2$ where:
 - $W_2 = (W_1 - F + 2P)/S + 1$
 - $H_2 = (H_1 - F + 2P)/S + 1$ (i.e. width and height are computed equally by symmetry)
 - $D_2 = K$
- With parameter sharing, it introduces $F \cdot F \cdot D_1$ weights per filter, for a total of $(F \cdot F \cdot D_1) \cdot K$ weights and K biases.
- In the output volume, the d -th depth slice (of size $W_2 \times H_2$) is the result of performing a valid convolution of the d -th filter over the input volume with a stride of S , and then offset by d -th bias.

Common settings:

Summary. To summarize, the Conv Layer:

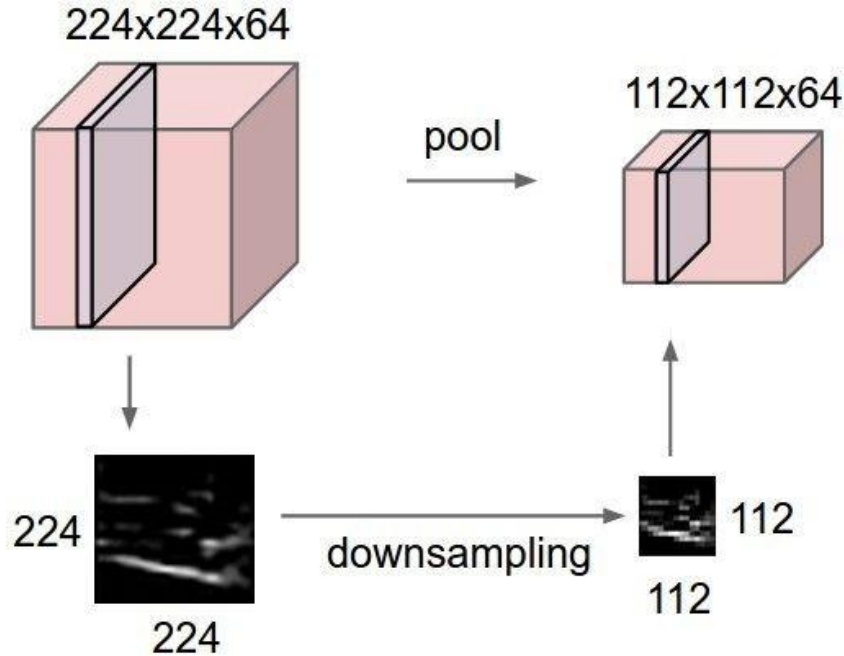
- Accepts a volume of size $W_1 \times H_1 \times D_1$
- Requires four hyperparameters:
 - Number of filters K ,
 - their spatial extent F ,
 - the stride S ,
 - the amount of zero padding P .
- Produces a volume of size $W_2 \times H_2 \times D_2$ where:
 - $W_2 = (W_1 - F + 2P)/S + 1$
 - $H_2 = (H_1 - F + 2P)/S + 1$ (i.e. width and height are computed equally by symmetry)
 - $D_2 = K$
- With parameter sharing, it introduces $F \cdot F \cdot D_1$ weights per filter, for a total of $(F \cdot F \cdot D_1) \cdot K$ weights and K biases.
- In the output volume, the d -th depth slice (of size $W_2 \times H_2$) is the result of performing a valid convolution of the d -th filter over the input volume with a stride of S , and then offset by d -th bias.

$K = (\text{powers of 2, e.g. 32, 64, 128, 512})$

- $F = 3, S = 1, P = 1$ (3x3 conv)
- $F = 5, S = 1, P = 2$ (5x5 conv)
- $F = 5, S = 2, P = ?$ (whatever fits)
- $F = 1, S = 1, P = 0$ (1x1 conv)
- $F = 3, S = 2, P = 1$ (Downs. by 2)

Pooling layer

- makes the representations smaller and more manageable
- operates over each activation map independently:



Hyperparameters:

Kernel Size

Stride

Pooling function

MAX POOLING

Single depth slice

x ↑

1	1	2	4
5	6	7	8
3	2	1	0
1	2	3	4

→ y

max pool with 2x2 filters
and stride 2



6	8
3	4

Introduces **invariance** to
small spatial shifts
No learnable parameters!

Common settings:

$$F = 2, S = 2$$

$$F = 3, S = 2$$

- Accepts a volume of size $W_1 \times H_1 \times D_1$
- Requires three hyperparameters:
 - their spatial extent F ,
 - the stride S ,
- Produces a volume of size $W_2 \times H_2 \times D_2$ where:
 - $W_2 = (W_1 - F)/S + 1$
 - $H_2 = (H_1 - F)/S + 1$
 - $D_2 = D_1$
- Introduces zero parameters since it computes a fixed function of the input
- Note that it is not common to use zero-padding for Pooling layers

two more layers to go: POOL/FC

